

The Scientific Discovery Engine

A from-scratch guide to the project: the science, the code, every function,
and what actually happened in Phases 1–8 and the real Gemini run

Generated from the project source and its recorded results

October 2026

Abstract

This guide assumes you know a little Python and a little calculus, and nothing else about the project. It first teaches the ideas the project is built from: partial differential equations, numerical derivatives, sparse regression, physics-informed neural networks, large language models, retrieval and knowledge graphs, and model selection. It then walks through the project phase by phase, explaining what was built, which functions do the work, what the experiments found, and what each phase taught. Every phase has a function-level flowchart, colour-coded by package. Appendix A lists every function in the codebase with a one-line description.

Contents

1	The big picture in plain words	3
1.1	The question the project answers	3
1.2	What the answer turned out to be	3
1.3	How the project was built: eight phases	3
1.4	How to read this guide	4
2	Background crash course	4
2.1	Fields and partial derivatives	4
2.2	Partial differential equations (PDEs) and the four mechanism families	5
2.3	Initial and boundary conditions, and well-posedness	5
2.4	Simulating a PDE: finite differences	6
2.5	Numerical derivatives from data, and why noise is the enemy	6
2.6	Sparse regression and SINDy	6
2.7	Neural networks, automatic differentiation, and training	7
2.8	Physics-informed neural networks (PINNs)	7
2.9	Large language models as hypothesis generators	7
2.10	Retrieval-augmented generation (RAG)	8
2.11	Knowledge graphs	8
2.12	Choosing between hypotheses: splits, parsimony, BIC	8
3	The whole machine on one page	9
3.1	How the code packages depend on each other	9

4	Phase 1: making the data	12
5	Phase 2: equation discovery by sparse regression	13
6	Phase 3: physics-informed neural networks	16
7	Phase 4: generating hypotheses with an LLM	17
8	Phase 5: scientific RAG and a knowledge graph	19
9	Phase 6: the physics-constrained generative search	20
10	Phase 7: evaluation, attacks and generalisation	23
11	The real LLM run with Google Gemini	25
11.1	How the switch from mock to Gemini works	26
11.2	Problems met, and the minimal fixes	26
11.3	Results	27
12	What is and is not demonstrated	28
13	Glossary	28
A	Complete function reference	29
B	Running the project yourself	39

1 The big picture in plain words

1.1 The question the project answers

Imagine a long thin metal rod. At the start, some parts are hot and some are cold. You cannot watch the whole rod continuously; instead, a few hundred cheap, noisy thermometer readings were taken at random places and random times. Each reading is just three numbers:

$$(x, t, u) = (\text{position, time, measured temperature}).$$

The question is: **what rule of nature produced these numbers?** Not “draw a curve through the points”, but “find the *law*”, the equation that says how u changes in time at every point, so that you could predict what happens at times and places nobody measured.

That is what this project calls *scientific discovery*: given sparse, noisy observations of a field $u(x, t)$, and *not* the governing equation, recover the equation, estimate its unknown constants, reject explanations that do not fit, and use the winner to predict outside the observed window.

Intuition: a detective, not a curve-fitter

A curve-fitter answers “what numbers come next?”. A detective answers “who did it, and how?”. This project is built like a detective agency:

- a **librarian** (RAG) fetches relevant textbook pages about how fields usually behave;
- an **archivist** (knowledge graph) keeps a structured card index of which mechanisms use which equations and parameters;
- a **brainstormer** (the LLM) suggests suspects: candidate equations;
- a **forensic lab** (physics-informed neural networks) tests each suspect against the evidence and estimates its unknown constants;
- a **judge** (model selection) picks the simplest suspect that fully explains the evidence;
- and a final **prediction test** checks the verdict on data that nobody used for any decision.

Crucially, the brainstormer is never allowed to decide the verdict. Only measured evidence decides.

1.2 What the answer turned out to be

The data was generated synthetically, so the true answer is known (but hidden from every decision):

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2}, \quad D = 0.1.$$

This is the *diffusion* (or heat) equation. In words: *each point drifts towards the average of its neighbours, at a speed set by D* . The project recovered this equation from 400 noisy points, estimated $\hat{D} = 0.10002$ (0.02% error), rejected the wrong candidates, and predicted the field beyond the observed time window with an error of 2.5×10^{-4} . Phase 7 then repeated the whole pipeline on a different, unseen system (advection–diffusion) and recovered both of its constants within 0.4%.

1.3 How the project was built: eight phases

The project grew in phases, each adding one capability and each *reporting its failures honestly*:

Phase	Adds	One-line takeaway
1	Synthetic data	Simulate diffusion, then sample 400 noisy points. A single-sine start turned out to be a trap.
2	Equation discovery (SINDy)	Regression on numerical derivatives finds the law on clean data, fails on noisy data.
3	PINNs	A neural network that must obey a candidate equation; it validates hypotheses and estimates D .
4	Hypothesis generation (LLM)	An LLM proposes equations as structured JSON; a compiler turns any equation into a PINN.
5	RAG + knowledge graph	Ground the LLM in retrieved textbook evidence, organised as a graph with provenance.
6	Physics-constrained search	The full pipeline: pool $\rightarrow$ filters $\rightarrow$ PINN per candidate $\rightarrow$ selection $\rightarrow$ OOD.
7	Evaluation	12 benchmark datasets, 95 questions, 20 adversarial attacks, ablations, a second system.
8	Final report	Audit of every claim: what was demonstrated and what was not.
+	Real LLM (Gemini)	The mock LLM was replaced by Google Gemini for a real run (Section 11).

1.4 How to read this guide

- **Section 2** teaches every concept from zero. If a word in a later section is unfamiliar, it is explained there.
- **Section 3** shows the whole machine on one page, plus a map of how the code packages depend on each other.
- **Sections 4–10** walk through the phases. Each has a function-level flowchart. Boxes are coloured by the package the function lives in (legend under each chart). Solid arrows mean “calls / passes data to”; dashed arrows mean “feeds back” or “used later”.
- **Section 11** covers the real Gemini run and compares it with the mock LLM.
- **Appendix A** is a complete function reference you can look things up in.

2 Background crash course

This section teaches, from zero, every idea the project uses. Each subsection ends with where the idea lives in the code.

2.1 Fields and partial derivatives

A **field** is a quantity that has a value at every place and time, written $u(x, t)$: the temperature along a rod, the concentration of dye in a pipe, the density of a population along a coastline. Here x is one-dimensional (a line), so the whole field can be drawn as a heat-map with x on one axis and t on the other.

A **partial derivative** measures how u changes when you nudge *one* variable and freeze the other. The project writes them with subscripts, exactly as the code does:

Code name	Maths	Meaning
u_t	$\partial u / \partial t$	how fast the value at a fixed point is changing in time
u_x	$\partial u / \partial x$	the slope of the profile at a fixed time (which way is “uphill”)
u_xx	$\partial^2 u / \partial x^2$	the curvature: positive in a valley, negative on a peak
u_xxx	$\partial^3 u / \partial x^3$	third derivative (in the library, never selected)

Intuition: curvature drives diffusion

On a peak, the neighbours are lower than the point, so the curvature u_{xx} is negative. In a valley the neighbours are higher and u_{xx} is positive. “Move towards the average of your neighbours” therefore means “change in time in proportion to the curvature”: $u_t = D u_{xx}$. Peaks sink, valleys fill, the profile smooths out.

2.2 Partial differential equations (PDEs) and the four mechanism families

A **PDE** is a rule connecting a field’s partial derivatives. All the equations in this project are *evolution equations*: they say what u_t is, given the current shape of u . The project works with four mechanism families (and combinations of them):

Family	Equation	What you would see
Diffusion	$u_t = D u_{xx}$	Peaks flatten and spread; sharp features vanish fastest. $D > 0$ is the diffusivity.
Advection	$u_t + c u_x = 0$	The whole shape slides sideways at speed c without changing shape.
Reaction (logistic)	$u_t = r u (1 - u/K)$	Each point grows (or decays) on its own towards a ceiling K ; neighbours do not interact.
Advection–diffusion	$u_t + c u_x = D u_{xx}$	The shape slides <i>and</i> spreads: smoke drifting in wind.
Reaction–diffusion	$u_t = D u_{xx} + r u (1 - u/K)$	Spreading plus local growth: a population expanding into new territory.

The letters D, c, r, K are **parameters**: unknown constants that must be estimated from data. A **hypothesis** in this project is an equation *shape* (which terms appear) together with its unknown parameters.

Key lesson: mechanism = which terms are present

Identifying the mechanism means identifying *which terms* the equation contains (u_{xx} ? u_x ? u and u^2 ?). Estimating parameters means finding their *numbers*. The project scores these two separately, because getting the numbers right does not prove you have the right terms (Section 10).

2.3 Initial and boundary conditions, and well-posedness

An evolution equation alone does not determine the future; you also need:

- an **initial condition** (IC): the profile at $t = 0$. Here $u(x, 0) = \sin(\pi x) + 0.5 \sin(3\pi x)$, a “double hump”;
- **boundary conditions** (BC): what happens at the ends of the rod. Here $u(0, t) = u(1, t) = 0$, the ends are held at zero (*Dirichlet* conditions).

A problem is **well-posed** if a solution exists, is unique, and changes only a little when the data changes a little. Two physical facts used later:

- Diffusion with $D < 0$ is the *backward heat equation*: it un-mixes, and tiny wiggles explode. It is ill-posed, so the project enforces the physical bound $D > 0$.
- Pure advection carries information in one direction only, so it can accept a boundary value only at the inflow end. Prescribing values at *both* ends over-determines it; the project warns about this.

2.4 Simulating a PDE: finite differences

Computers cannot handle a continuum, so the rod is cut into grid points x_i spaced Δx apart and time into steps Δt . Derivatives become differences of neighbouring values:

$$u_{xx}(x_i) \approx \frac{u_{i+1} - 2u_i + u_{i-1}}{\Delta x^2}, \quad u_i^{n+1} = u_i^n + \Delta t D \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}.$$

The second formula is the **explicit (forward-time, centred-space) scheme** used to generate the data in Phase 1. It is only stable if $D \Delta t / \Delta x^2 \leq 1/2$, which is why the simulation uses 800 time steps for 61 grid points. Later phases also need a general solver for *any* candidate equation (`validation/forward_solver.py`); it offers simple Euler steps and the more accurate fourth-order Runge–Kutta (RK4) method. Euler with centred differences is unconditionally unstable for advection, which is why RK4 was added in Phase 7.

2.5 Numerical derivatives from data, and why noise is the enemy

To *discover* an equation from data you need the derivatives of the data. With scattered points you first **reconstruct** a regular grid by interpolation, optionally **smooth** it, then take finite differences. The trouble is that differencing amplifies noise. If each value has noise of size σ , the second-difference formula divides by Δx^2 , so its error is roughly

$$\text{error in } u_{xx} \sim \frac{\sigma}{\Delta x^2},$$

which becomes enormous on a fine grid. A third derivative is worse still. Smoothing reduces the noise but introduces bias. This single fact explains why Phase 2 works on clean data and fails on noisy data, and why the project later turned to PINNs, which never take numerical differences of the data.

2.6 Sparse regression and SINDy

SINDy (Sparse Identification of Nonlinear Dynamics) turns “find the equation” into a regression:

1. Compute u_t at many points: this is the target vector y .
2. Build a **library** Θ whose columns are candidate terms evaluated at the same points. Here: $1, u, u_x, u_{xx}, u_{xxx}, u^2, uu_x, uu_{xx}$ (8 columns).
3. Find coefficients ξ with $y \approx \Theta \xi$, while forcing most entries of ξ to be exactly zero, because real laws have few terms.

The project’s sparsifier is **STLSQ** (Sequentially Thresholded Least Squares): fit by ordinary least squares, set every coefficient smaller than a threshold (0.05) to zero, refit using only the surviving columns, and repeat until nothing changes. Columns are normalised first so the threshold means the same thing for every term.

Watch out: identifiability and the eigenfunction trap

Regression can only separate terms whose columns look different. For $\sin(\pi x)$, the second derivative is $u_{xx} = -\pi^2 u$ *exactly*, so the u and u_{xx} columns are perfectly proportional. No method can tell $D u_{xx}$ apart from $-D\pi^2 u$. Phase 2 fell into exactly this trap with the original Phase 1 data and “discovered” $u_t = -0.987 u$: a perfect fit, with the wrong physics. Using two sine modes with different curvatures fixed it.

2.7 Neural networks, automatic differentiation, and training

A **multilayer perceptron** (MLP) is a function built from layers: each layer multiplies its input by a weight matrix, adds a bias, and applies a squashing function (here $\tanh$). The project’s network takes the two numbers (x, t) and outputs one number $u_\theta(x, t)$, where θ stands for all the weights. It has four hidden layers of 64 units.

Training means adjusting θ to make a loss (an error score) small, using its gradient. The project uses **Adam** (3000 steps, a robust first-order optimiser) and then **L-BFGS** (300 iterations, a quasi-Newton method that converges much further once Adam is close).

Automatic differentiation (`autograd`, `torch.autograd.grad`) gives *exact* derivatives of the network’s output with respect to its inputs. So $\partial u_\theta / \partial t$ and $\partial^2 u_\theta / \partial x^2$ come for free, with no finite differences and no noise amplification. This is the key to PINNs.

2.8 Physics-informed neural networks (PINNs)

A **PINN** is a neural network trained to do two things at once: pass close to the observations *and* obey a candidate equation everywhere. For a candidate written as a residual $\mathcal{R} = 0$ (for diffusion, $\mathcal{R} = u_t - D u_{xx}$) the loss is

$$\mathcal{L} = \underbrace{\frac{1}{N} \sum_i (u_\theta(x_i, t_i) - u_i)^2}_{\text{data: fit observations}} + \underbrace{\frac{1}{M} \sum_j \mathcal{R}(x_j, t_j)^2}_{\text{physics: obey the equation}} + \underbrace{\mathcal{L}_{\text{BC}}}_{\text{ends held at 0}} + \underbrace{\mathcal{L}_{\text{IC}}}_{\text{start profile}}.$$

The physics term is evaluated at random **collocation points** (x_j, t_j) that need no measurements. All four weights are 1 in this project.

- **Forward PINN:** the parameters are known (e.g. $D = 0.1$ fixed). The PINN acts as a smart interpolator/denoiser that respects physics.
- **Inverse PINN:** the parameters are unknown. D becomes a trainable number, optimised together with θ . Whatever D lets the network fit data *and* physics best is the estimate.

Intuition: why a PINN can reject a wrong equation

If a candidate equation is wrong, no network can satisfy both the data and that equation at the same time; one of the loss terms stays large. If it is right, all terms can be driven to near zero. So the final losses of a trained inverse PINN are *evidence* for or against the hypothesis, and the fitted parameters are estimates.

2.9 Large language models as hypothesis generators

A **large language model** (LLM) such as Google Gemini predicts text. Given a description of what was observed (“the amplitude decreased substantially, the peak did not move...”), it can propose plausible equations. The project treats the LLM strictly as a *brainstormer*:

- it must answer in **structured JSON** matching a schema (**Hypothesis:** id, name, equation, parameters, required derivatives, mechanism, assumptions, rationale, confidence);

- its **confidence** field is recorded but *never* used as evidence;
- a deterministic validator checks syntax, variables, parameters and derivatives *before* any expensive PINN training;
- the verdict comes only from measured PINN losses.

For testing without internet access, a **mock provider** returns a fixed, canned set of answers. All Phase 4–7 numbers come from the mock; Section 11 reports the real Gemini run.

2.10 Retrieval-augmented generation (RAG)

RAG means: before asking the LLM, retrieve relevant documents and include them in the prompt, so its proposals are grounded in evidence. The steps:

1. **Corpus:** five short Markdown “textbook” documents, one per mechanism.
2. **Chunking:** split each document at its section headers into chunks.
3. **Embedding:** turn each chunk into a vector. The project uses a simple, deterministic *bag-of-words hashing* embedding: each word is hashed into one of 256 slots, slot counts form the vector, and the vector is normalised to length 1.
4. **Search:** embed the query (the observation description) the same way and return the chunks with the highest *cosine similarity* (the dot product of unit vectors).

Answer leakage is the danger that retrieved text gives away the answer. The corpus states only generic textbook laws (“Fick’s second law, $u_t = D u_{xx}$ ”) and never this dataset’s value $D = 0.1$; an automated check verifies this.

2.11 Knowledge graphs

A **knowledge graph** stores facts as typed nodes and typed edges: *Mechanism* “diffusion” GOVERNS *Equation* “ $u_t = D u_{xx}$ ”, which HAS_PARAMETER *Parameter* “ D ”, SUPPORTED_BY *Document* “diffusion.md”. Every edge records its **provenance** (which document and chunk it came from). Where RAG answers “what text is relevant?”, the graph answers structured questions such as “which parameters appear in the documented equations for diffusion?” (answer: D). In this project the graph has 22 entities and 40 relationships, all sourced, and it supplies *equation templates* that join the LLM’s proposals in the candidate pool.

2.12 Choosing between hypotheses: splits, parsimony, BIC

Data splits. To judge a model honestly you must test it on data it never saw:

- **train:** used to fit (observations with $t \leq 0.8$, minus a holdout);
- **validation (held-out):** a random 20% of the $t \leq 0.8$ points, used for the accept/reject *gate* and for selection;
- **OOD (out-of-distribution):** all points with $t > 0.8$, never used to train or select. It measures genuine extrapolation into the future.

Nested models. Reaction–diffusion with $r = 0$ *is* diffusion; advection–diffusion with $c = 0$ *is* diffusion. A more general model can always fit at least as well, so “best fit” alone favours needless complexity. This is the *nested-model problem*.

Parsimony (Occam’s razor). Among models that fit equally well, prefer the simplest. The project makes this precise:

1. *Gate*: reject a candidate unless its held-out error, physics residual, BC/IC losses and physical bounds are all within fixed thresholds.
2. *Equivalence*: keep the gate survivors whose held-out RMSE is within 5% of the best.
3. *Complexity*: choose the lowest score = $\# \text{terms} + \# \text{free parameters} + \text{max derivative order} + 2 \cdot \# \text{nonlinear terms} + 0.1 \cdot \# \text{operations}$.
4. *BIC* as a tie-breaker and cross-check: $\text{BIC} = n \ln(\text{MSE}) + k \ln n$, which rewards fit and charges $\ln n$ per parameter.

Term necessity. For each parameter, measure what share of u_t (RMS) its terms contribute on the trained PINN. Below 5% the parameter is *inactive*: the data does not need that term. Phase 7 promoted this into the selection rule (Section 10).

The oracle rule. Because the data is synthetic, the clean truth is known. It may be used *only* to report error *after* every decision has been made (the function `oracle_evaluation`), never to make a decision. Phase 7 audits the code automatically for violations of this rule.

Where it lives in the code

Derivatives: `equation_discovery/derivatives.py` (finite differences) and `pinn/derivatives.py` (autograd). SINDy: `equation_discovery/sindy.py`. PINN: `pinn/`. LLM: `llm/`. RAG: `rag/`. Graph: `knowledge_graph/`. Selection: `validation/model_selection.py`, `hypotheses/complexity.py`. Splits and the gate: `hypotheses/search.py`.

3 The whole machine on one page

Figure 1 is the complete discovery pipeline as finally assembled in Phase 6. Read it top to bottom: each stage hands its output to the next. The right-hand column says which package does the work and which question the stage answers.

Key lesson: who is allowed to decide

- RAG *retrieves*, the graph *organises*, the LLM *proposes*. None of them can mark an equation as correct; there is literally no field for that in the schema.
- Ranking (`hypotheses/ranking.py`) only sets the *order* in which candidates are examined.
- Only measured numbers decide: the gate (`validate_candidate`) and `select_model`, both using held-out *noisy* observations, never the hidden truth.
- Status strings are deliberately modest: “supported by the available observations and physics constraints” or “rejected under the tested conditions”. Never “proven”.

3.1 How the code packages depend on each other

Figure 2 was computed from the actual `import` statements in the code. An arrow $A \rightarrow B$ means “package *A* imports from package *B*”. Double-headed arrows are mutual imports.

Intuition: reading the map

`hypotheses` sits in the middle because it glues the brainstormer (`llm`) to the lab (`pinn`) and the judge (`validation`). `evaluation` (Phase 7) reaches into almost everything because it re-runs the whole pipeline on benchmark datasets. `data` and `pinn` import each other only because the problem description and the PINN share the same data-loading helper (`load_observational_data`).

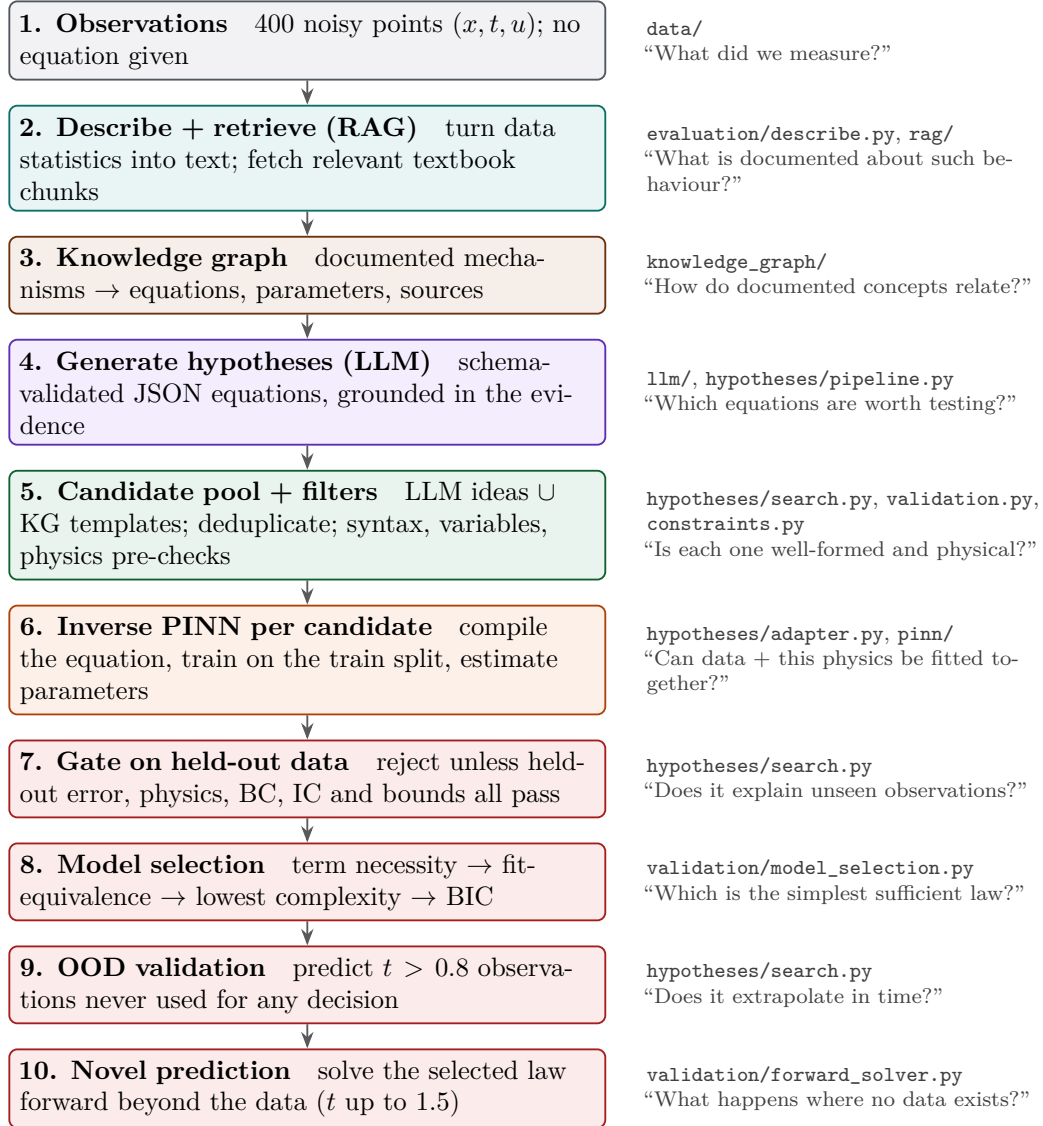


Figure 1: The end-to-end discovery pipeline (Phase 6 form). Colours follow the package legend used in every flowchart.

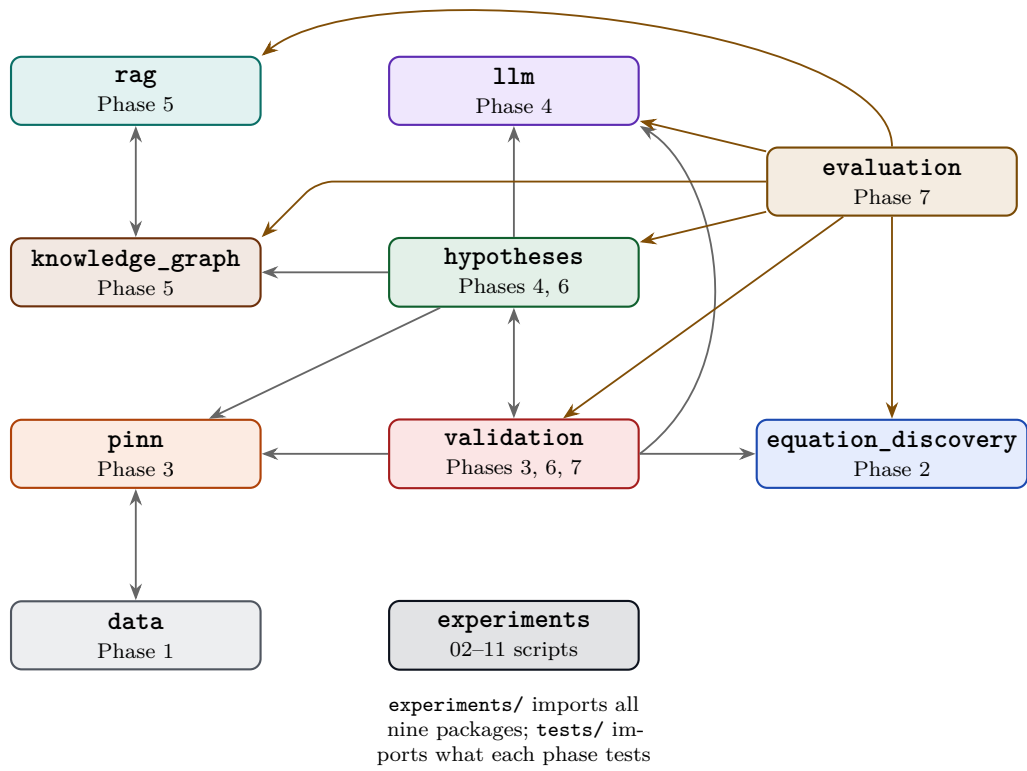


Figure 2: Package dependency map, derived from the real imports. The phase that introduced each package is shown under its name.

4 Phase 1: making the data

Goal. Produce realistic observations of a known law, so that every later phase can be graded against a truth it is not allowed to see.

What happens. `data/generate_diffusion.py` simulates $u_t = 0.1 u_{xx}$ on $x \in [0, 1]$, $t \in [0, 1]$ with the explicit finite-difference scheme (61 grid points $\times$ 800 time steps), starting from the double hump $\sin(\pi x) + 0.5 \sin(3\pi x)$ with both ends held at zero. From the full field it saves three files:

<code>full.npz</code>	the entire clean field (800×61). For grading and plots only.
<code>sparse.npz</code>	a noise-free random 10% of grid points (4880 points). For ablations of sparsity alone.
<code>noisy.npz</code>	400 random points with Gaussian noise (std = 5% of the field's std, ≈ 0.0116). The only file the discovery system may see , and it uses only its x, t, u columns.

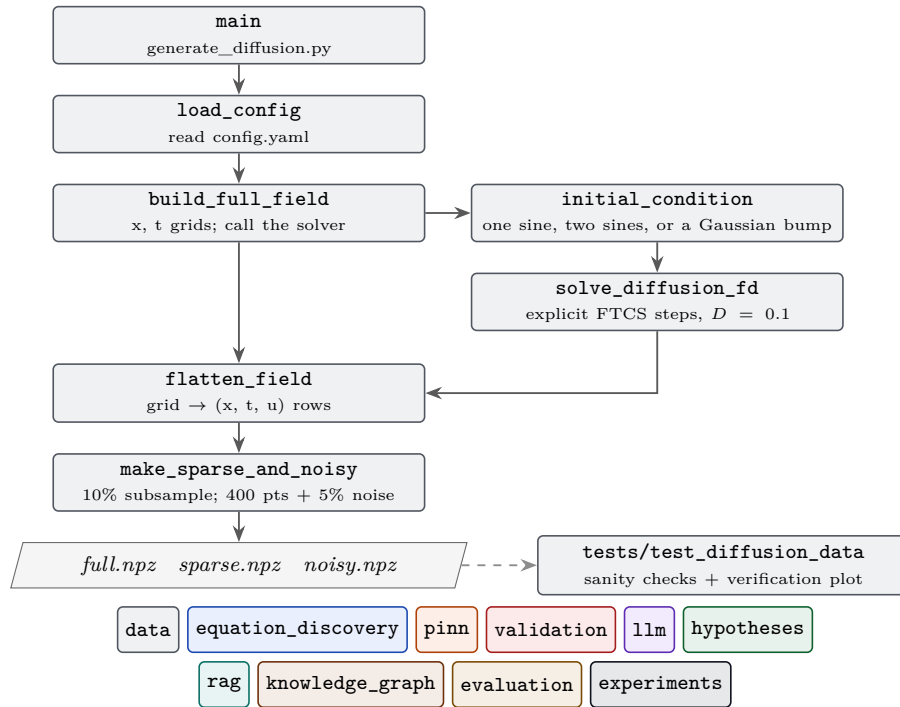


Figure 3: Phase 1 function flow: configuration $\rightarrow$ simulation $\rightarrow$ sampling $\rightarrow$ files $\rightarrow$ checks.

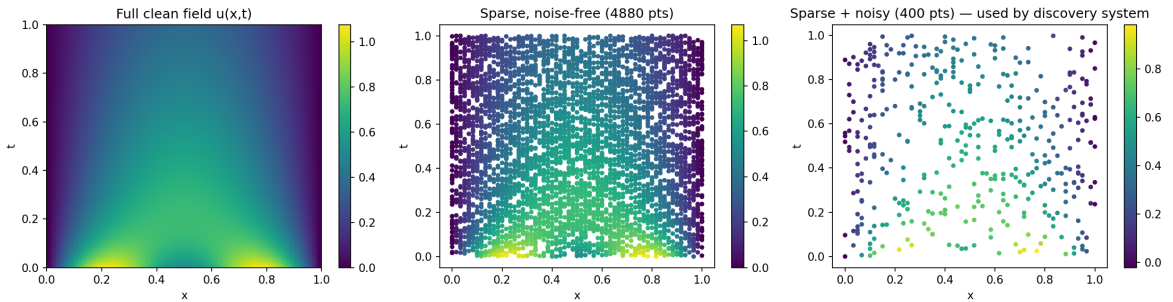


Figure 4: Phase 1 verification plot (from the project's results): the clean field (left), the sparse sample, and the 400 noisy points the system is allowed to see. The double hump decays and merges into one broad hump, because the faster-wiggling mode $\sin(3\pi x)$ decays 9 times faster than $\sin(\pi x)$.

Key lesson: the eigenfunction trap

The first version started from a single $\sin(\pi x)$. That turned out to be undiscoverable: for this profile $u_{xx} = -\pi^2 u$ exactly, so “ $D u_{xx}$ ” and “ $-D\pi^2 u$ ” are the same signal and no method can separate them. The two-mode start mixes two different curvatures and breaks the tie. The single-mode option is kept in the code as a documented cautionary example, and Phase 7’s adversarial suite re-tests it (case A13).

5 Phase 2: equation discovery by sparse regression

Goal. Recover $u_t \approx D u_{xx}$ from data alone using SINDy (Section 2.6), and report honestly where it works and where it does not.

How it works. For clean data (already a grid), take finite differences directly. For scattered data, first reconstruct a grid by linear interpolation (41×150 nodes for 4880 sparse points, but only 13×25 for 400 noisy points), optionally smooth it with a Gaussian filter, then difference. Build the 8-column library and run STLSQ with threshold 0.05.

Results.

Data	Discovered equation	$\hat{D}$ (error)	Right terms?
Clean	$u_t = 0.1009 u_{xx}$	0.1009 (0.9%)	yes
Sparse	$u_t = 0.1013 u_{xx}$	0.1013 (1.3%)	yes
Noisy, no smoothing	$0.50 - 3.42u + 4.15u^2 + \dots$ (5 terms)	–	no
Noisy, smoothed	$0.55u + 0.20u_{xx} - 0.51u^2 + \dots$ (5 terms)	–	no

On noisy data the free 8-term search fails. But the evidence is still there: fitting single-term laws gives diffusion $R^2 = 0.79$ ($\hat{D} = 0.107$) while advection has *negative* R^2 (worse than predicting the mean) on every dataset. So advection is correctly disfavoured even when the unrestricted search is confused.

Key lesson: a good fit is not the right law

Noisy SINDy produced equations with small residuals and wrong terms. Differentiating noisy data amplifies the noise (Section 2), and regression then explains noise with extra terms. This motivates Phase 3: test hypotheses with a method that never differentiates the data.

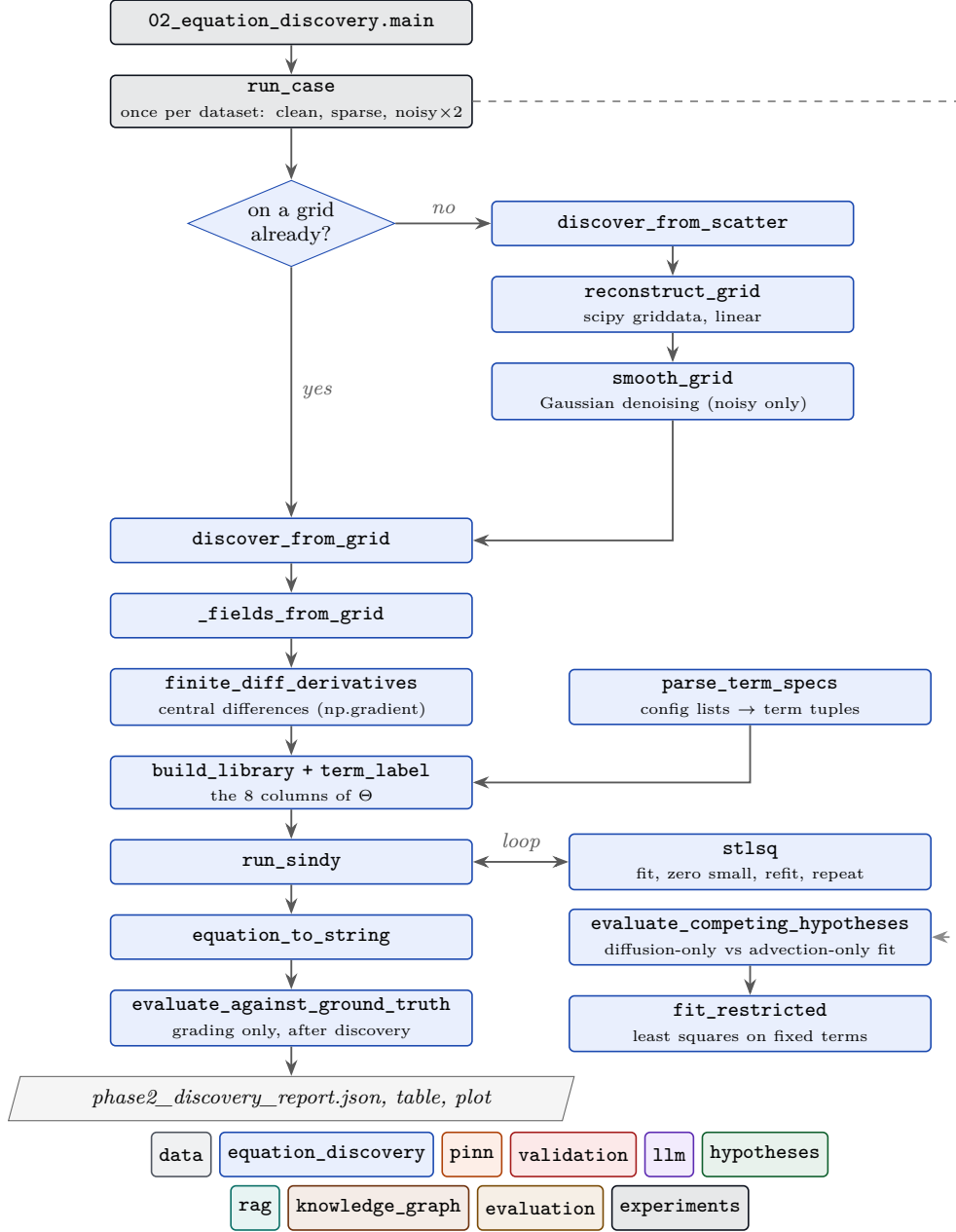


Figure 5: Phase 2 function flow. Scattered data is first put on a grid; then derivatives, library, sparse regression. The competing-hypothesis check (right) fits each single-term law separately.

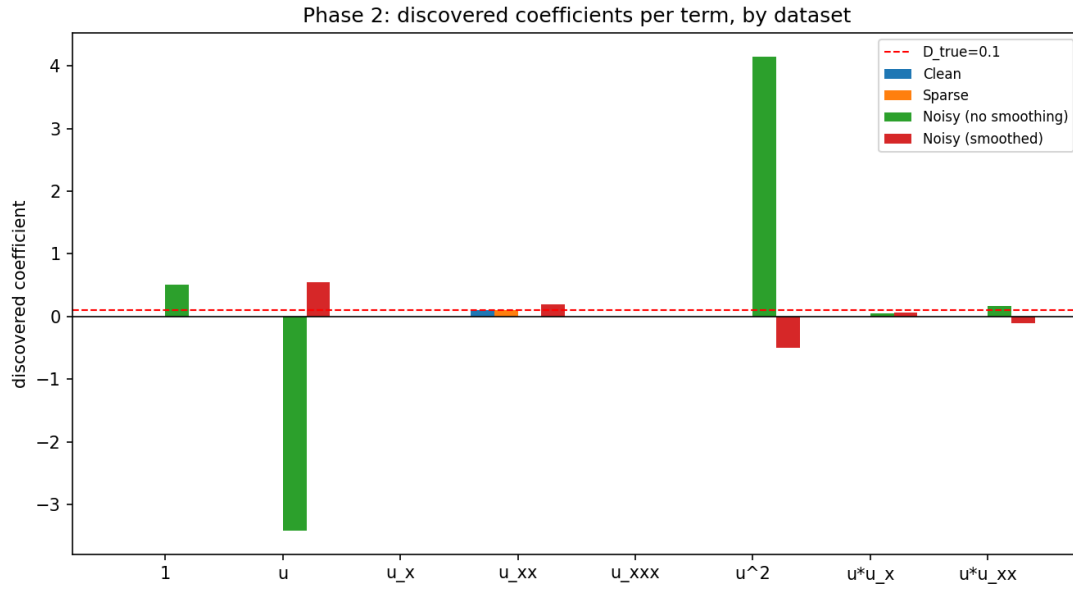


Figure 6: Phase 2 coefficients found by STLSQ for each run (from `results/equations/`). Clean and sparse data keep only u_{xx} ; noisy data scatters weight over spurious terms.

6 Phase 3: physics-informed neural networks

Goal. Build one reusable PINN (Section 2.8) and use it three ways: forward (known D), inverse (estimate D), and hypothesis validation (diffusion vs. advection).

How it works. The network PINN maps $(x, t) \mapsto u$ with four tanh layers of 64 units. The *only* thing that changes between hypotheses is the residual function, looked up in `RESIDUAL_REGISTRY` (diffusion, advection, reaction–diffusion). All derivatives come from autograd (`compute_derivatives`). Training draws 2000 collocation points, 200 boundary and 200 initial points, runs 3000 Adam steps then 300 L-BFGS iterations, and finally scores the result against fixed thresholds.

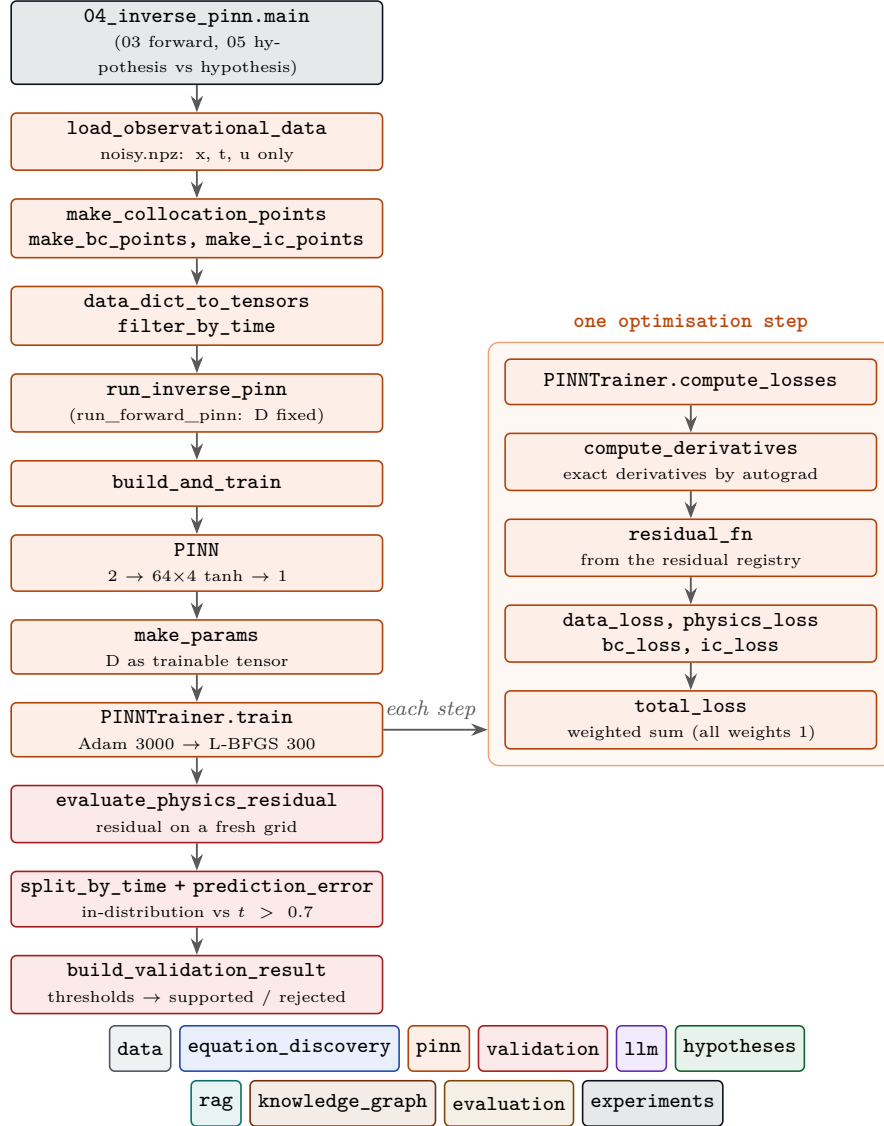


Figure 7: Phase 3 function flow. Left: an experiment builds tensors, creates the network and trains it. Right: what happens inside every optimisation step. Bottom: scoring with fixed thresholds.

Results.

- **Forward** ($D = 0.1$ known): the PINN’s prediction differs from the clean field by RMSE 0.000727, even though it was trained on data with ≈ 0.0116 noise. The physics term denoises.
- **Inverse** (D unknown, started at 0.2): $\hat{D} = 0.09989$, a 0.11% error. D slides from 0.2 to 0.1 within about 1500 Adam steps and stays there.
- **Validation**, both trained on $t \leq 0.7$ only with their parameter trainable:

Hypothesis	Status	Data loss	Physics	IC loss	In-dist. RMSE	OOD RMSE
H1 diffusion	supported	0.00013	0.00008	0.000001	0.00078	0.00115
H2 advection	rejected	0.03775	0.00499	0.02515	0.20033	0.33145

Diffusion wins on every metric by 30–290 $\times$. Advection simply cannot turn a decaying double hump into anything consistent with the data: its IC loss alone is 25,000 times worse.

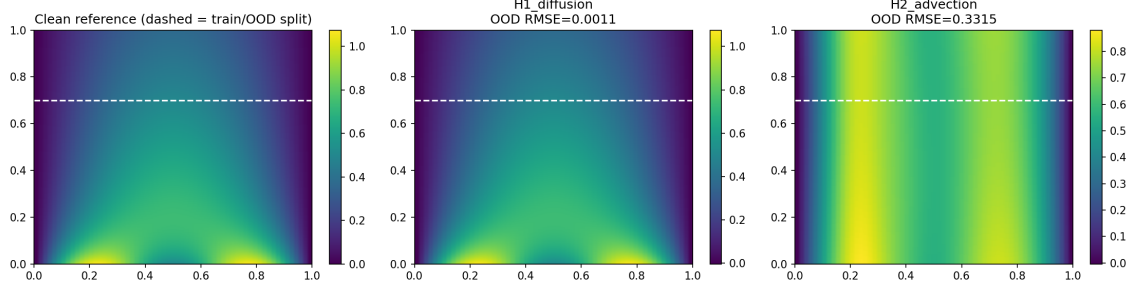


Figure 8: Phase 3 hypothesis validation (from `results/pinn/`). Training used only $t \leq 0.7$ (dashed line). Diffusion reproduces the field on both sides of the split; advection converges to a wrong, roughly static pattern.

Watch out: an audit note added in Phase 6

The Phase 3 accept/reject rule included one check against the clean truth field. Training never used it, but a decision did. Re-checking showed no outcome depended on it (H2 also fails its data and IC losses). Phase 6 removed the issue: from then on, decisions use only held-out *noisy* observations.

7 Phase 4: generating hypotheses with an LLM

Goal. Until now a human typed each candidate equation into the code. Phase 4 adds a generator that proposes *which* equations are worth testing, from a description of what was observed, without being told the answer. The rule that makes this safe: *LLM = hypothesis generator*, *PINN = validator*.

The description the LLM sees. `build_problem_description` computes only statistics of the noisy observations and turns them into text. For the Phase 1 data it says, among other things: the peak amplitude is about 0.983 near $t = 0.05$ and about 0.395 near $t = 0.95$ (“decreased substantially”); the field stays near zero at the edges; and the centroid moves only from $x \approx 0.53$ to 0.41 (“little evidence of bulk translation”). It never names a mechanism.

The key design choice: a generic equation compiler. Phase 3 needed a hand-written Python function per equation. That does not scale to whatever an LLM might propose. `compile_residual` instead parses the equation *string* with SymPy (a symbolic-maths library) and compiles it into exactly the residual function the Phase 3 trainer expects. For example, from `"u_t = D*u_xx + r*u*(1-u/K)"` it detects the fields u, u_t, u_{xx} and the parameters D, r, K automatically. Any combination of $u, u_t, u_x, u_{xx}, u_{xxx}$ and named parameters works with zero code changes.

Deterministic checks before any training (`validate_hypothesis`):

1. *Syntax*: can SymPy parse the equation?
2. *Variables*: is u the dependent variable and are x, t the independent ones?
3. *Parameters*: does every symbol used appear in the declared parameter list, and vice versa?
4. *Derivatives*: do the LLM’s stated `required_derivatives` match what SymPy finds?

5. *Units*: what fraction of parameters declare units (reported, not a proof)?
6. *Duplicates*: **canonical_signature** replaces every parameter by a placeholder and ignores sign and side, so $u_t = D u_{xx}$ and $k u_{xx} = u_t$ are recognised as the same equation.

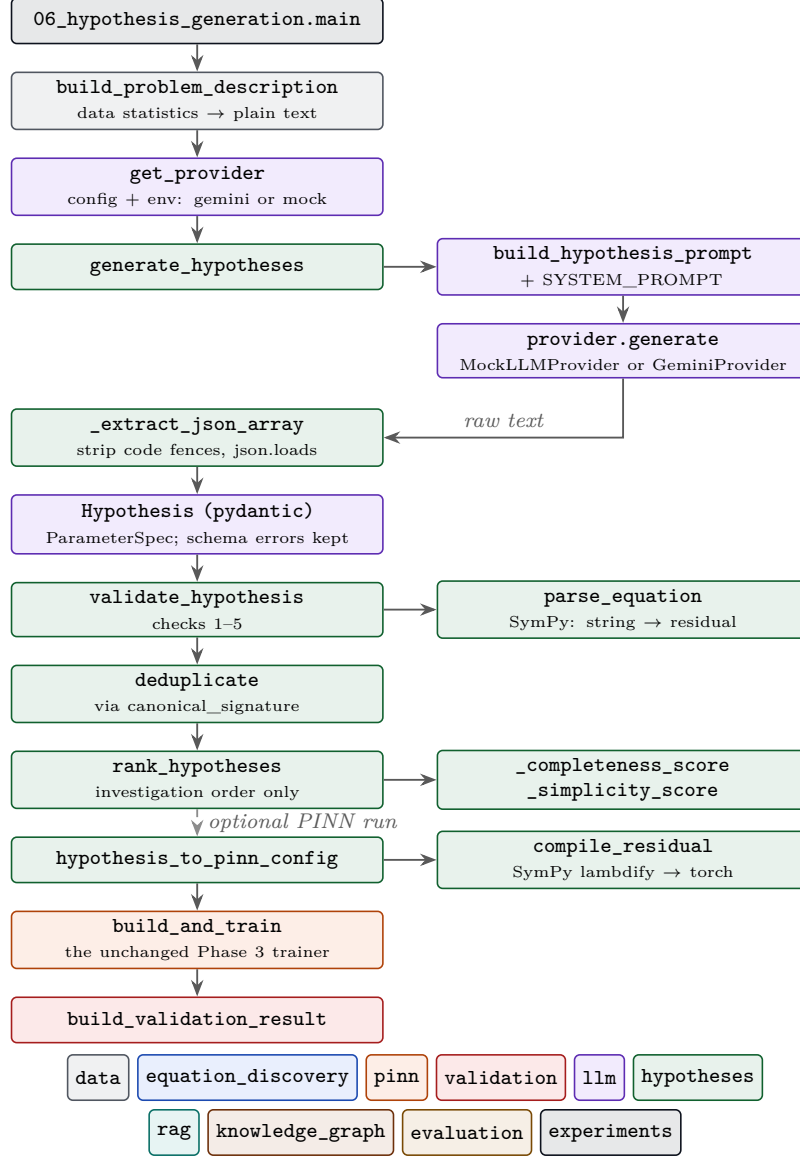


Figure 9: Phase 4 function flow: describe → prompt → LLM → parse JSON → schema → deterministic checks → deduplicate → rank → (optionally) compile into a PINN and validate.

Results (mock LLM, all four candidates run through the Phase 3 PINN).

ID	Equation	PINN status	Fitted parameters
H1	$u_t = D u_{xx}$	supported	$D = 0.09989$
H2	$u_t + c u_x = 0$	rejected	$c = 0.0164$
H3	$u_t = D u_{xx} + r u(1 - u/K)$	supported	$D = 0.1006, r = 0.0148, K = 1.215$
H4	$u_t + c u_x = D u_{xx}$	supported	$c = -0.0004, D = 0.0999$

Key lesson: validation alone cannot reject harmless extra terms

H3 and H4 *contain* diffusion as a special case. Their extra parameters converged to almost exactly the “switch this term off” values ($r \approx 0.015$, $c \approx 0$), so they fit as well as H1 and pass the same thresholds. A pass/fail validator cannot tell “minimal correct model” from “needlessly general model that also fits”. This is the **nested-model problem** (Section 2.12), and it is why Phase 6 adds term necessity, complexity and BIC to the selection.

8 Phase 5: scientific RAG and a knowledge graph

Goal. Ground hypothesis generation in retrieved scientific evidence (RAG), and organise that evidence as a knowledge graph with provenance, so every fact can be traced to its source.

The corpus. Five short Markdown documents in `rag/corpus/`: diffusion, advection, reaction, reaction–diffusion and advection–diffusion. Each has a front-matter header (id, title, source, domain, declared mechanism, declared equations, declared parameters) and textbook-style sections.

Preventing answer leakage. Documents state only the generic textbook law (e.g. “Fick’s second law, $u_t = D u_{xx}$ ”). They never state this dataset’s $D = 0.1$ and never say “this data is diffusion”. A real scientist knows the textbook candidates before looking at new data; knowing the answer would be cheating. An automated check confirms the string “0.1” appears in no corpus document.

Results.

- The graph has **22 entities** (5 documents, 5 mechanisms, 3 variables, 4 parameters, 5 equations) and **40 relationships, all sourced**. Shared concepts merge: there is a single node for D , even though three documents declare it, and repeated facts merge their sources instead of creating duplicate edges.
- Example queries: asking `equations_for_mechanism` about diffusion returns $u_t = D * u_{xx}$, and asking `parameters_for_mechanism` returns D .
- For the Phase 1 data, the context contained the mechanisms advection, diffusion and reaction, their three textbook equations, and the parameters D, K, c, r .
- **Mode A** (no context) and **Mode B** (with context) produced the identical four candidate equations. That is expected: the mock LLM returns canned answers whatever the prompt. Showing that evidence changes the proposals needs a real LLM (Section 11).

Key lesson: RAG and the graph do different jobs

RAG answers “which text is relevant?” by similarity; it is fuzzy and ranked. The graph answers “how are these documented concepts related, and who said so?”; it is exact and traceable. Neither decides what is true. Phase 7 measures their real contribution and finds it modest: RAG gives 100% recall of the right mechanism on a 5-document corpus, but top-1 retrieval is right only 1 time in 6.

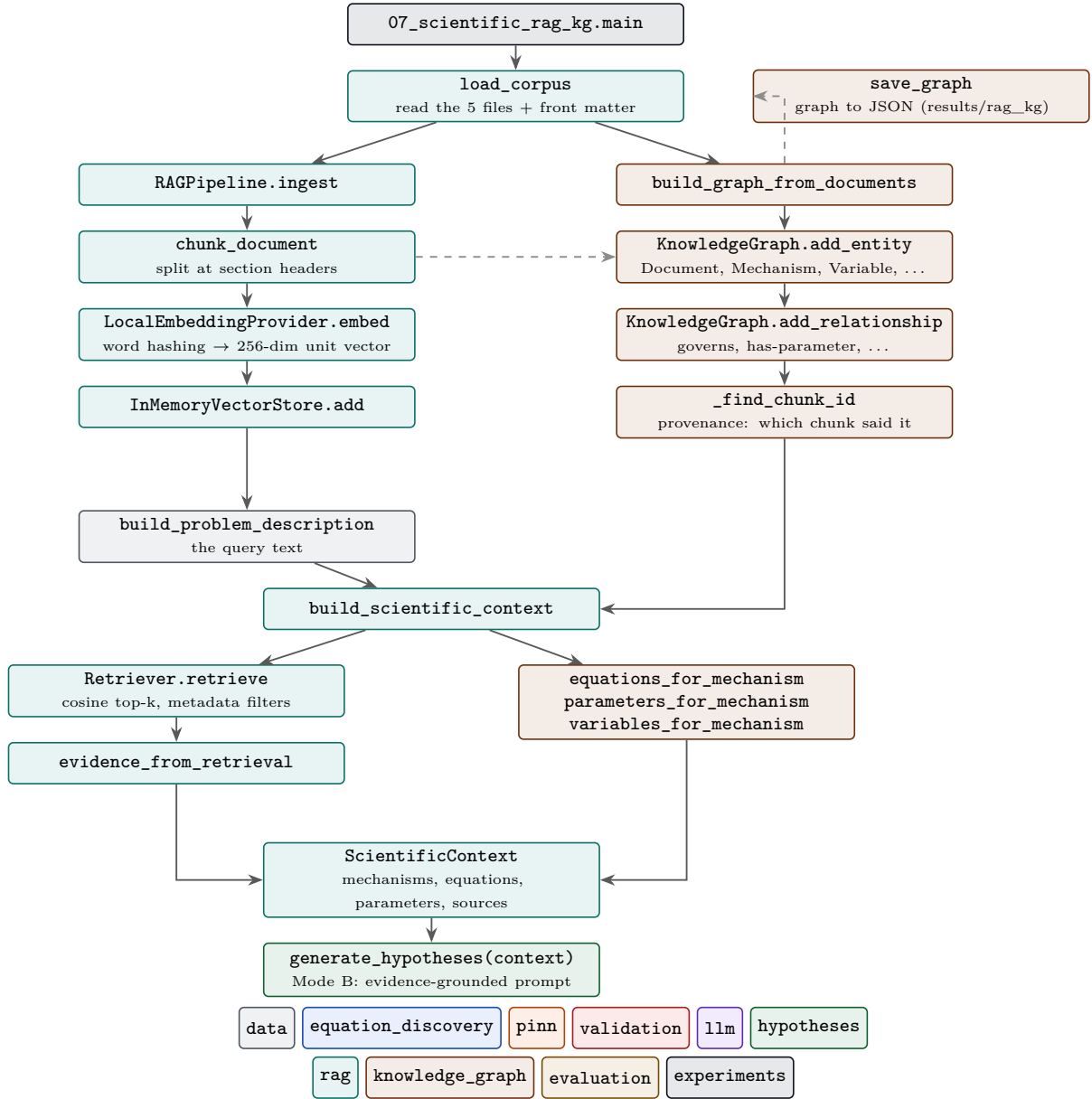


Figure 10: Phase 5 function flow. Left: RAG indexing (chunk, embed, store). Right: knowledge-graph construction with provenance (the dashed arrow: chunks give each relationship its source). Bottom: the observation description is used as a query; retrieved evidence and graph queries merge into one `ScientificContext` that is passed to the hypothesis generator.

9 Phase 6: the physics-constrained generative search

Goal. Assemble everything into one honest discovery pipeline (Figure 1) that searches over candidate equations, tests each with an inverse PINN, and selects the most parsimonious supported law, *without the truth influencing any decision*.

Three stages. Each inverse PINN takes about 4 minutes on a CPU, so the experiment runs in stages and caches every candidate. The script is `experiments/08_physics_constrained_search.py`:

1. **stage_prepare:** RAG, graph, context, LLM generation, knowledge-graph templates, deduplication, structural and physics pre-checks. No training.
2. **stage_pinn:** one inverse PINN per surviving candidate, on the training split only.
3. **stage_select:** stability, model selection, baselines, OOD and novel prediction, then the oracle

report and plots.

The splits. Train on $t \leq 0.8$ (minus a random 20% holdout); gate and select on that holdout; test extrapolation on all $t > 0.8$ points, which no decision ever sees. Collocation, BC and IC points are also restricted to $t \leq 0.8$, so OOD prediction is genuine extrapolation.

Physics constraints. Each parameter can carry bounds; `config.yaml` supplies fallbacks $D > 0$ and $K > 0$ (c and r may have either sign). Before training, `precheck_physics` rejects non-evolution equations and warns on two real modelling issues: advection with fixed values at both ends is over-determined, and an equation with no spatial derivative cannot interact with boundary conditions at all.

Results (mock LLM). The pool held four LLM candidates plus the graph templates. The graph’s diffusion and advection templates were recognised as duplicates of H1 and H2, but the graph contributed one genuinely new candidate the LLM had not proposed: pure reaction (K3).

ID	Equation	Estimates	Holdout RMSE	Status	Inactive
H1	$u_t = D u_{xx}$	$D = 0.10002$	0.01214	selected	–
H2	$u_t + c u_x = 0$	$c = 0.0089$	0.2269	rejected	–
H3	$u_t = D u_{xx} + r u(1 - u/K)$	$D = 0.101, r = 0.023, K = 1.18$	0.01217	supported	r, K
H4	$u_t + c u_x = D u_{xx}$	$c = -0.0002, D = 0.09995$	0.01214	supported	c
K3	$u_t = r u(1 - u/K)$	$r = -1.44, K = 6.67$	0.1176	rejected	–

H1, H3 and H4 all sit at the noise floor (≈ 0.012), so they are fit-equivalent; H3 and H4 have inactive extra terms and reduce to H1; H1 has the lowest complexity, and BIC agrees independently. Note that H4 had the *lowest raw* holdout error (by 0.06%). A pure best-fit rule would have picked the more complex model.

Out-of-distribution and novel prediction (columns marked “oracle” are evaluation only):

Model	OOD RMSE, noisy obs.	OOD RMSE, clean (oracle)
H1 PINN (selected)	0.0114	0.0018
H2 PINN (rejected)	0.3249	0.3337
Data-only MLP (no physics)	0.0727	0.0712
Forward solve of the selected law	0.0113	0.0003

Solved forward to $t \in (1.0, 1.5]$, where no data exists at all, the selected law has oracle RMSE 2.5×10^{-4} . The physics-free network is about 40 times worse out of distribution.

Watch out: a disagreement reported, not hidden

The bootstrap stability check (`bootstrap_stability`) refits terms by least squares on finite-difference derivatives, a cheap proxy, not PINN retraining and not Bayesian. For H3 it finds sign-stable u and u^2 coefficients, while the PINN finds the reaction term contributes under 1%. The same spurious terms appear in Phase 2’s noisy SINDy failure, which points to a finite-difference artefact. The report flags this instead of choosing the convenient answer.

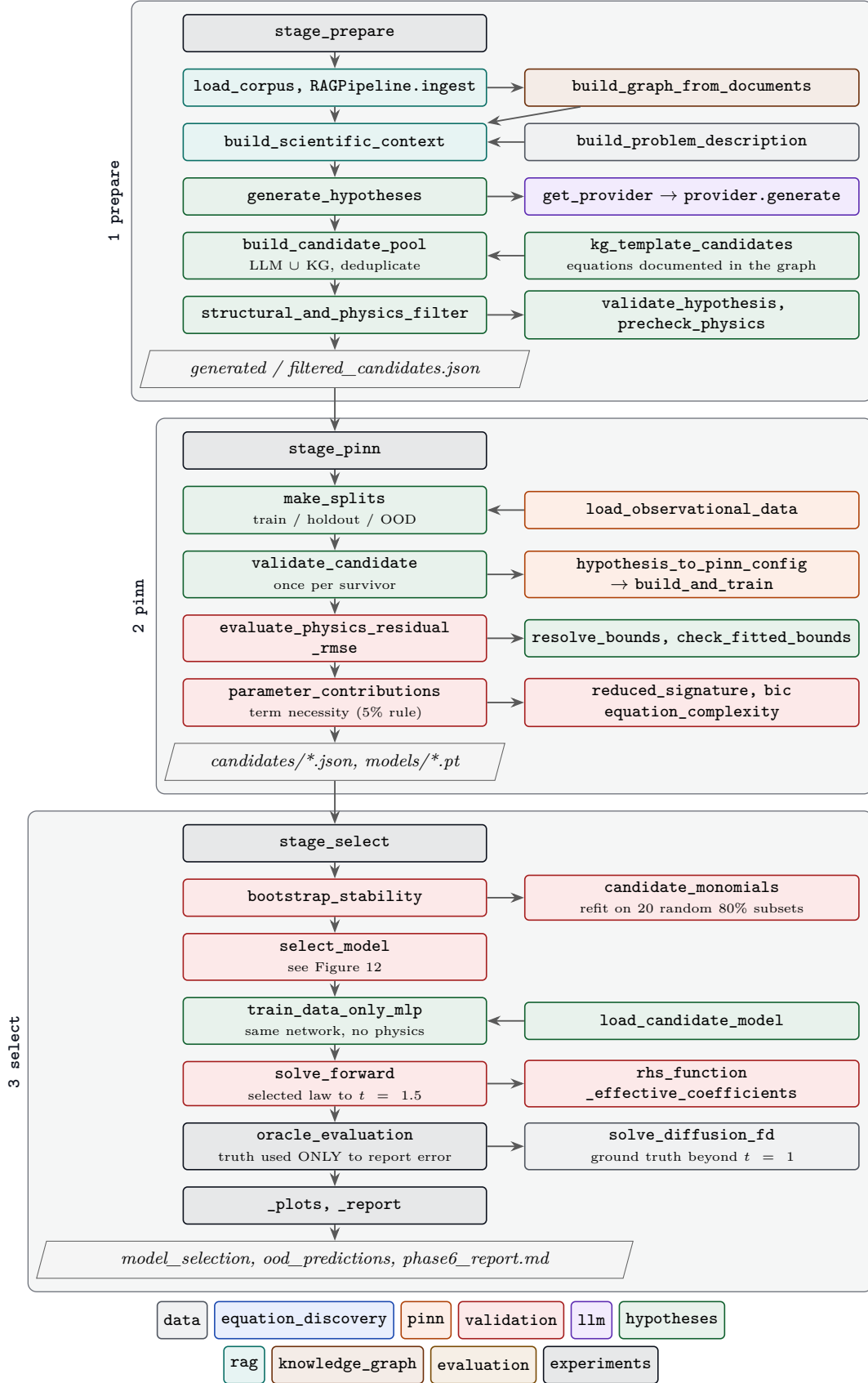


Figure 11: Phase 6 function flow, stage by stage. The left spine is the order of execution; the right column holds the helper functions each step calls. Only `oracle_evaluation` ever touches the ground truth, and it runs after every decision is made.

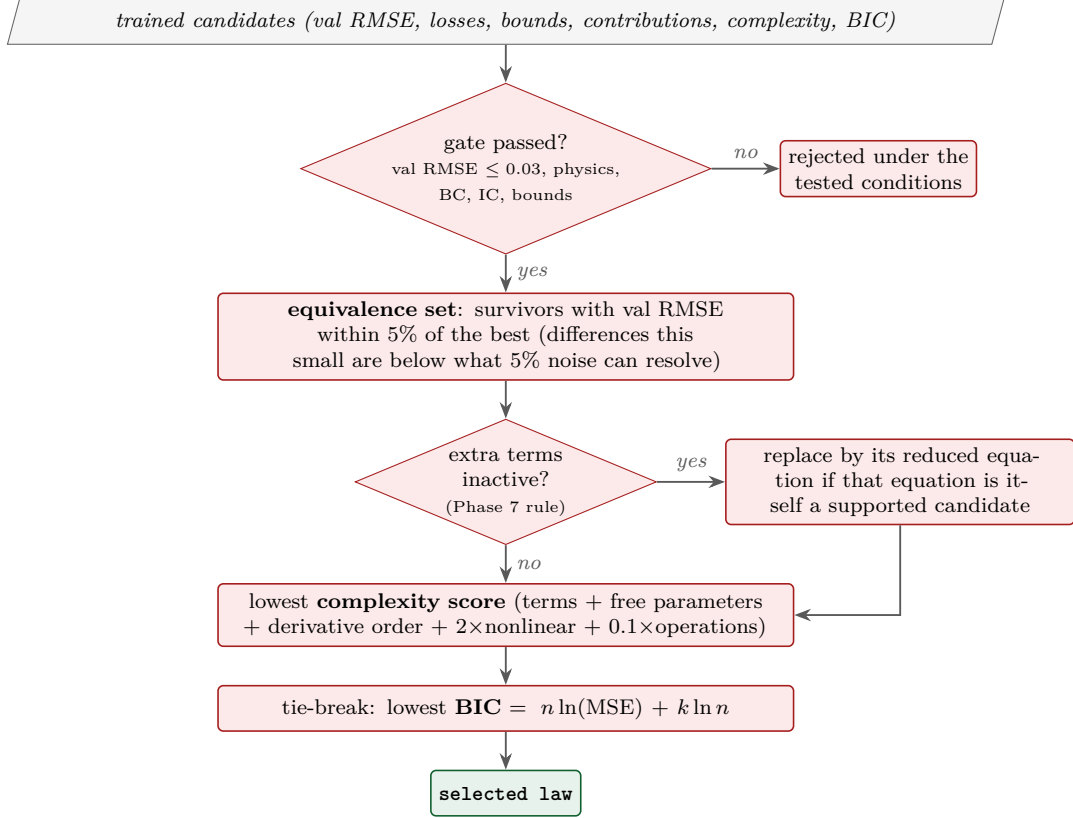


Figure 12: The selection rule inside `select_model`, with thresholds fixed in `config.yaml` before any run. The “inactive terms” step is the opt-in Phase 7 refinement (`prefer_reduced`); Phase 6 used only the term-necessity numbers to *explain* its choice.

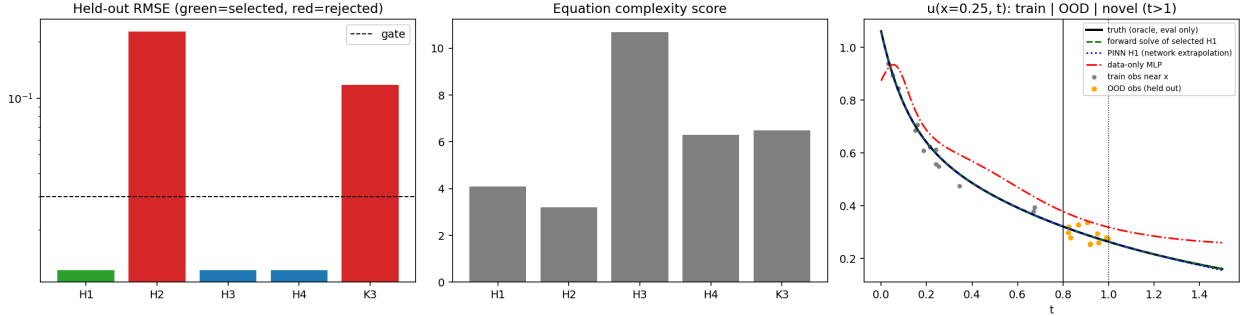


Figure 13: Phase 6 summary (from `results/phase6/`): held-out error per candidate with the gate line (left), complexity scores (middle), and a time trace at $x = 0.25$ through train, OOD and novel windows (right).

10 Phase 7: evaluation, attacks and generalisation

Goal. One success on one system proves little. Phase 7 asks: does the pipeline work across mechanism families? Can it be fooled? Which components actually matter? Does it generalise to a new system?

What was built (the `evaluation/` package, no new architecture):

- `datasets.py`: 12 benchmark datasets, 4 families $\times$ 3 parameter settings, using exact analytic solutions where they exist and RK4 otherwise.
- `describe.py`: turns any dataset into the text query for RAG with one fixed template; a test

checks it never names a mechanism.

- `engine.py`: the Phase 6 pipeline with the PINN swapped for fast least squares on finite-difference derivatives, so 95 items run in about a minute. (The PINN version runs separately on a second system.)
- `identify.py`, `benchmark.py`, `adversarial.py`, `ablation.py`, `leakage.py`.

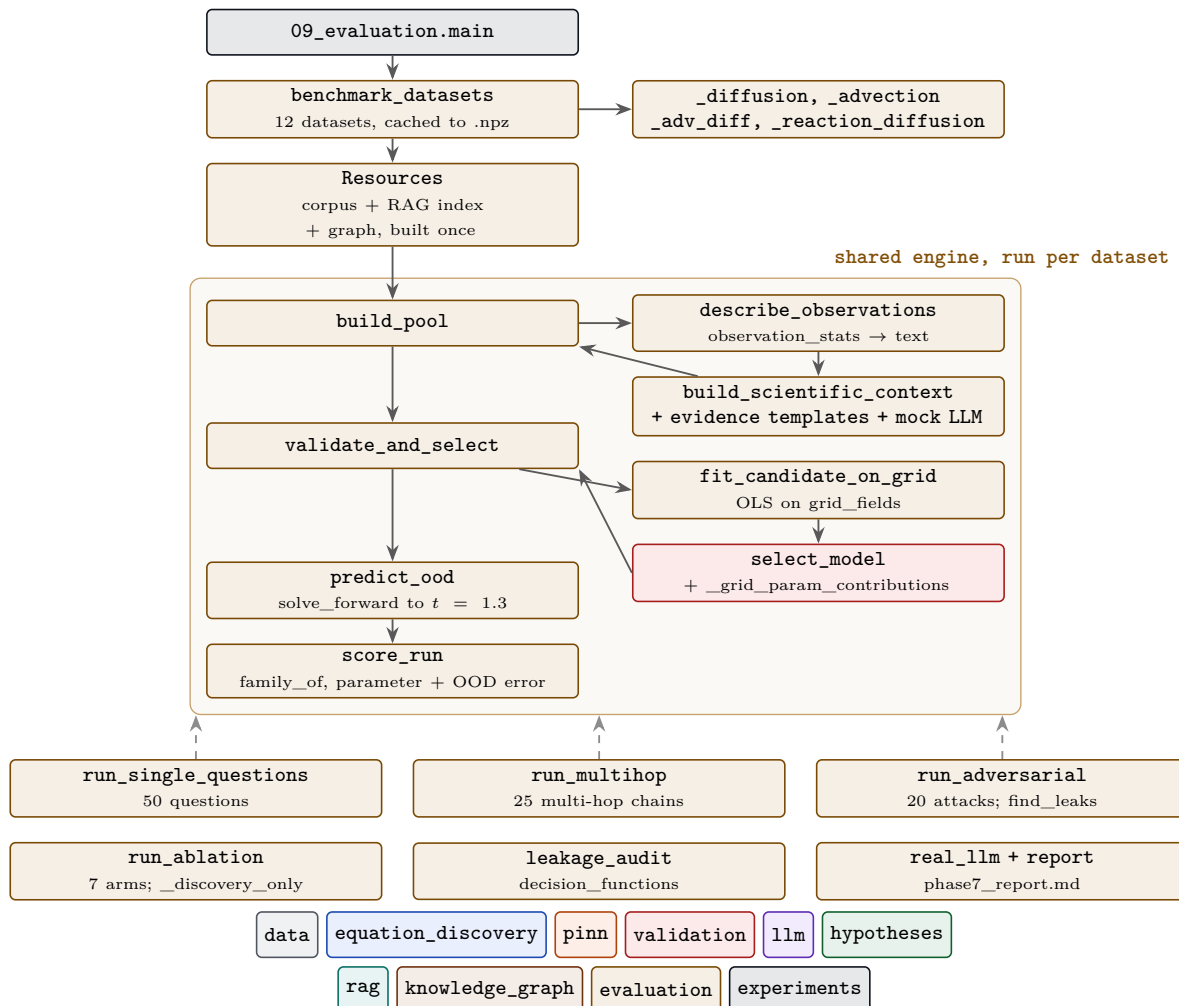


Figure 14: Phase 7 function flow. A shared engine (shaded box) runs the pipeline on each benchmark dataset; the six suites (bottom) call it in different ways. `10_generalization.py` separately runs the full PINN pipeline (`validate_candidate`, `select_model`) on a new system.

Results

50 questions: 50/50. Structure identification 12/12, parameters within 5% 12/12, plus graph, compiler, complexity and retrieval-recall questions. Caveat: several categories are easy by construction, and **top-1 retrieval is right only 1 time in 6**.

25 multi-hop problems: 23/25. The longest chains (data → description → RAG → graph → validation → selection → beyond-window prediction, 6 hops) score 12/12. Both failures come from top-1 lexical retrieval choosing “advection” for combined mechanisms.

20 adversarial attacks: 20/20. Examples: undeclared symbols, missing time derivative, negative diffusivity, *time-reversed data whose best fit needs $D < 0$* (rejected by the physical bound), disguised duplicates, malicious LLM output, the eigenfunction trap (flagged non-identifiable), a simpler-but-wrong candidate on reaction–diffusion data (rejected at 85% error rather than chosen for simplicity),

corpus answer leakage and prompt injection. Three noise/sparsity cases pass *by abstention*: no candidate clears the gate, which is safe but is not discovery.

Ablation: which components matter? (correct model out of 12 datasets)

Arm	Correct
SINDy alone	8/12 (advection–diffusion 0/3)
RAG + generation, no validation	3/12 (always “diffusion”)
RAG + graph + generation, no validation	3/12
Full system	12/12
Full system with the original Phase 6 rule	10/12
Full system without the LLM	12/12
Retrieved-evidence equations + validation (no graph, no LLM)	12/12

Key lesson: what each part really contributes

Physics validation and selection produce the correct answers; without them, generation just repeats its favourite guess. RAG supplies a candidate pool that contains the truth. On this benchmark, the graph and the *mock* LLM add no measurable accuracy; the graph’s value is structured querying and provenance. SINDy is fast and accurate when every true term is large, but its threshold silently drops small real terms. And **accurate prediction is not the same as the right law**: the Phase 6 rule predicted OOD well on 12/12 while identifying only 10/12 laws correctly.

The disclosed rule change. On noise-free two-mode diffusion, finite-difference u_{xx} gives each sine mode a slightly wrong decay rate, and the extra u term of reaction–diffusion corrects both modes exactly, so that model fits the *discretisation error* to 10^{-14} and wins “within 5% of best”. Yet its reaction term contributes only 0.27% of the dynamics. Phase 7 therefore promoted term necessity into the rule (the “inactive?” step in Figure 12), as an opt-in flag so Phase 6 results stay reproducible. It changes 2 of 12 benchmark outcomes and never strips a real reaction term.

Generalisation to a new system with the full PINN pipeline. Hidden truth: $u_t + 0.4 u_x = 0.02 u_{xx}$ on $x \in [0, 1.5]$, Gaussian start, 400 points with 5% noise.

Candidate	Fitted	Holdout RMSE	Status	OOD vs clean (oracle)
diffusion	$D = 0.075$	0.096	rejected	0.111
advection	$c = 0.369$	0.064	rejected	0.113
reaction–diffusion	$D = 0.117, r = 1.22, K = 1.44$	0.084	rejected	0.095
advection–diffusion	$c = 0.39897, D = 0.02008$	0.0099	selected	0.0010
data-only MLP	–	0.0124	–	0.0068

Both constants within 0.4% of the truth; solving the law forward beyond the data gives 0.58% relative error. SINDy on the same noisy data returned a spurious equation.

Leakage audit. A static code check confirms none of the six decision functions reference ground-truth names; none of the 15 hidden benchmark values appears in the corpus; and the audit catches a deliberately planted cheating selector, so the check is not vacuous.

11 The real LLM run with Google Gemini

Every LLM result in Phases 4–7 came from the deterministic *mock* provider, so the project could not claim anything about LLM-driven discovery. After the project was assembled, a real provider for Google Gemini was added and the Phase 6 pipeline was run with it.

11.1 How the switch from mock to Gemini works

- `config.yaml` sets `llm.provider: "gemini"`. `get_provider` returns a `GeminiProvider` only if the environment variable `GEMINI_API_KEY` is set; otherwise it warns and falls back to the mock.
- The key is read from the environment and never written anywhere. It lives in a local `.env` file, which is listed in `.gitignore`.
- `experiments/11_real_llm_gemini.py` reuses the existing Phase 6 and Phase 7 code unchanged, refuses to run if the key is missing (so it can never silently use the mock), and writes only to `results/evaluation/real_llm_gemini/`. The mock results were verified byte-for-byte unchanged.

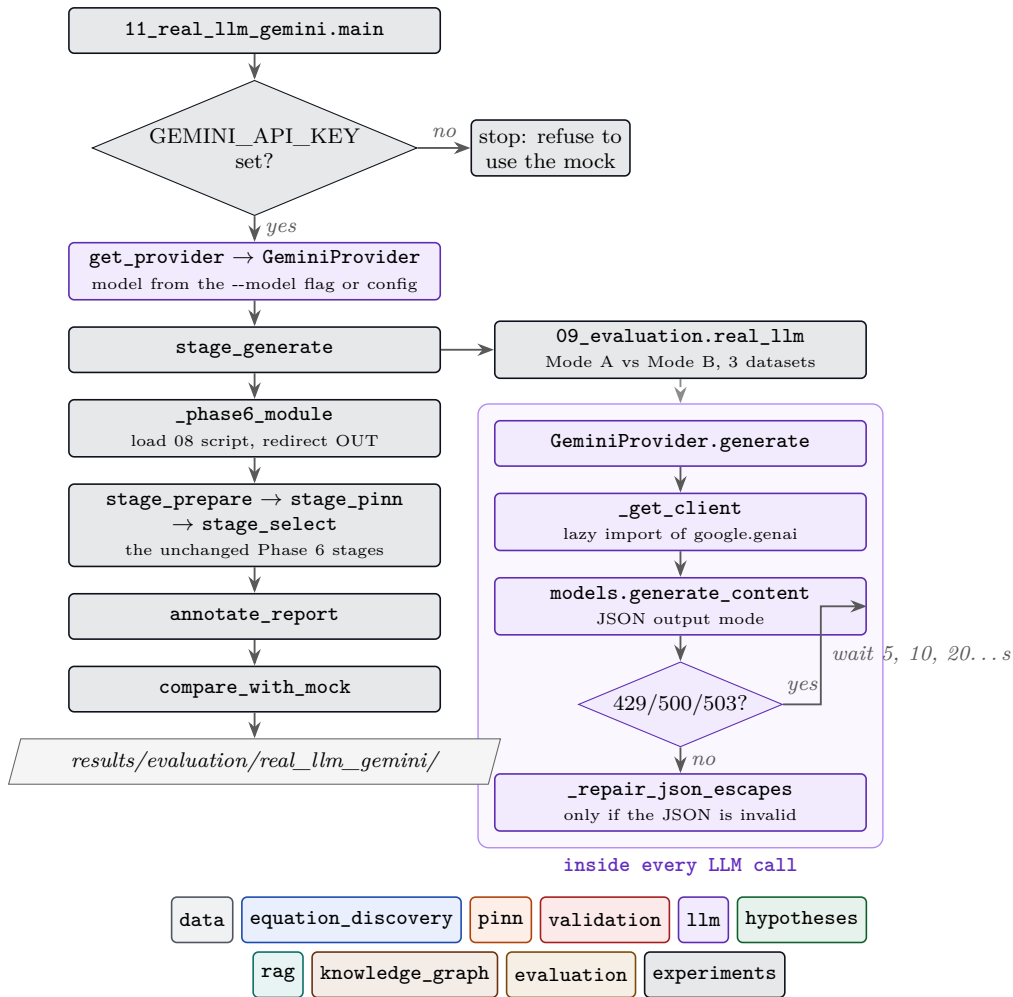


Figure 15: The real-LLM run. Left: the runner reuses the existing stages. Right: what `GeminiProvider.generate` does on every call, including the two compatibility fixes added for Gemini.

11.2 Problems met, and the minimal fixes

1. **Overload.** The configured `gemini-3.8-flash` (and `gemini-3.5-flash`) repeatedly answered *503: model is currently experiencing high demand*. The provider now retries 429/500/503 with increasing waits. When that was still not enough, the run used `gemini-3.5-flash-lite`, passed with `--model` for that run only; the configuration was not changed.

2. **Invalid JSON.** Gemini sometimes wrote L^AT_EX such as `\partial` inside JSON strings. That is an illegal escape, and the strict parser threw away the *entire* response. The fix asks Gemini for its native JSON mode and, if the text still fails to parse, doubles stray backslashes.

Neither fix touched the scientific pipeline. Many Gemini proposals were still (correctly) rejected by the project’s own validator, mostly because Gemini writes powers as u^3 where the compiler expects $u**3$, or uses u_{xxx} , which is not in the derivative set.

11.3 Results

Did evidence (RAG) change Gemini’s hypotheses? Yes, on all three test datasets.

Dataset	Mode A (no evidence)	Mode B (with retrieved evidence)
diffusion	decay and logistic variants only; no plain diffusion	plain diffusion $u_t = D u_{xx}$ appears
reaction–diffusion	correct family $D u_{xx} + r u(1 - u/K)$	no valid hypothesis (one response failed schema)
advection–diffusion	correct family, velocity named v	same family, renamed to the corpus’s c

Full Phase 6 pipeline with Gemini (same data, splits and thresholds as the mock run):

	Mock (Phase 6)	Gemini (gemini-3.5-flash-lite)
LLM candidates	$D u_{xx}$; $c u_x$ advection; reaction–diff.; adv.–diff.	$D u_{xx}$; $D u_{xx} - r u$; adv.–diff.; reaction–diff.
Graph templates added	K3 (pure reaction)	K1 (pure advection), K3
Passed the gate	H1, H3, H4	H1, H2, H3, H4
Rejected	H2 (advection), K3	K1 (advection), K3
Selected	$u_t = D u_{xx}$	$u_t = D u_{xx}$
$\hat{D}$ (error)	0.10002 (0.021%)	0.10001 (0.011%)
Holdout RMSE	0.01214	0.01218
OOD RMSE, noisy obs.	0.01144	0.01160
OOD RMSE vs clean	0.00181	0.00201
field (oracle)		
Novel window $t \in (1, 1.5]$ (oracle)	2.5×10^{-4}	2.7×10^{-4}

Gemini’s three more complex candidates all fit as well as diffusion, but their extra terms were judged inactive ($r \approx 0.013$, $c \approx 0.001$, $r \approx 0.017$), each reduces to plain diffusion, and the same law won on complexity and BIC. The real LLM proposed more physically plausible alternatives than the mock’s canned pure advection, and the pipeline reached the same, correct conclusion.

Watch out: one sample

LLM output is random from run to run. This is a single Gemini sample on one dataset; it shows the pipeline works with a real LLM and that evidence changes the proposals, not that Gemini reliably discovers laws.

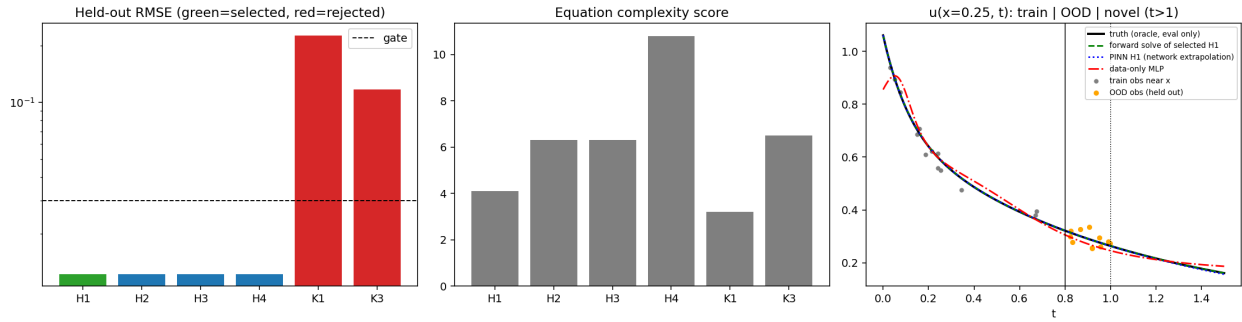


Figure 16: The Gemini run’s Phase 6 summary (from `results/evaluation/real_llm_gemini/phase6/`), in the same layout as the mock run’s figure.

12 What is and is not demonstrated

The project’s final report audits every claim. In short:

Claim	Verdict	Why
PINN forward and inverse	demonstrated	D and c recovered within 0.4% on two systems
Physics-constrained search	demonstrated	Phase 6 and the second system
OOD and novel prediction	demonstrated	both systems; physics-free network 6–40× worse
Adversarial robustness	demonstrated	20/20 (3 by abstention)
LLM-driven discovery	partial (new)	one real Gemini run; same correct answer as the mock
RAG contribution	partial	recall only; top-1 retrieval 1/6
Knowledge-graph contribution	partial	structured queries and provenance; no accuracy gain
Noisy-data robustness	partial	PINN path works at 5% noise; SINDy fails
Uncertainty	partial	bootstrap only, not Bayesian; disagrees with the PINN on H3
Equation uniqueness	not demonstrated	nested models fit equally; selection is a stated parsimony rule
Boundary/initial-condition inference	not demonstrated	supplied as known physics
Real-world data; a new law	not demonstrated	synthetic 1D data; known laws recovered

Main limitations. Synthetic one-dimensional PDEs only; boundary and initial conditions are given, not inferred; lexical (word-hashing) retrieval over a five-document corpus; hand-set thresholds (fixed in advance, but not calibrated on independent data); the fast regression engine has no noise robustness; and uncertainty is a bootstrap, not a posterior.

13 Glossary

Ablation

Removing one component at a time to measure how much it contributes.

Abstention

Declining to select any law when no candidate passes the gate. Safe, but not discovery.

Autograd

Automatic, exact differentiation of a neural network’s output, used for u_t, u_x, u_{xx} in PINNs.

BIC

Bayesian information criterion, $n \ln(\text{MSE}) + k \ln n$: fit quality plus a penalty per parameter.

Canonical signature

An equation’s structure with parameter names and signs normalised, for duplicate detection.

Collocation points

Random (x, t) points where a PINN’s equation residual is penalised; no data is needed there.

Complexity score

Terms + free parameters + derivative order + $2 \times$ nonlinear terms + $0.1 \times$ operations.

Embedding

A vector representing a piece of text, so similar texts get similar vectors.

Finite differences

Approximating derivatives from neighbouring grid values; amplifies noise.

Forward / inverse problem

Forward: parameters known, predict u . Inverse: estimate unknown parameters from data.

Gate

The pass/fail check on held-out error, physics residual, BC, IC and physical bounds.

Hypothesis

A candidate equation with its unknown parameters, in the `Hypothesis` JSON schema.

Identifiability

Whether the data can distinguish two terms at all (the eigenfunction trap is a failure of it).

Knowledge graph

Typed entities and typed relationships, each with provenance.

Leakage

Any route by which the hidden answer could influence a decision.

Nested model

A model that contains another as a special case (reaction–diffusion with $r = 0$ is diffusion).

OOD

Out-of-distribution: here, observations later than $t = 0.8$ that no decision ever sees.

Oracle

The hidden truth, used only to report error after all decisions.

PINN

Physics-informed neural network: a network trained to fit data and obey an equation.

RAG

Retrieval-augmented generation: retrieve relevant documents and include them in the LLM prompt.

Residual

For an equation written as $\mathcal{R} = 0$, the value of $\mathcal{R}$; zero when the equation holds.

SINDy / STLSQ

Sparse regression for equation discovery / its thresholded least-squares solver.

Term necessity

The share of u_t a parameter’s terms contribute; under 5% means inactive.

A Complete function reference

Every function and class in the project (tests excluded), grouped by package and module, generated from the source code so that nothing is missed. Methods are listed as `Class.method`. In total: 284 entries in 70 modules.

data/

data/generate_diffusion.py

load_config	Read config.yaml into a dict.
initial_condition	The starting profile $u(x,0)$: one sine (degenerate), two sines (default) or a Gaussian bump.
solve_diffusion_fd	Explicit finite-difference solver for $u_t = D u_{xx}$ with zero boundary values; produces the ground truth.
build_full_field	Build the x and t grids from config and run the solver: the clean 800×61 field.
flatten_field	Turn the 2-D field into matching 1-D arrays (x, t, u), one row per grid point.
make_sparse_and_noisy	Draw the 10% noise-free subsample and the 400 noisy observations (5% noise).
main	Phase 1 entry point: generate and save full.npz, sparse.npz and noisy.npz.

data/problem_description.py

build_problem_description	Compute statistics of the noisy observations (amplitude change, edges, centroid, smoothness) and write them as plain text, never naming a mechanism.
---------------------------	--

equation_discovery/

equation_discovery/derivatives.py

reconstruct_grid	Interpolate scattered (x, t, u) samples onto a regular grid (scipy griddata, nearest-neighbour at edges).
smooth_grid	Gaussian smoothing of a reconstructed grid to reduce noise before differentiation.
finite_diff_derivatives	Central finite differences on a grid: $u_t, u_x, u_{xx}, u_{xxx}$, with edge cropping.

equation_discovery/discovery_engine.py

_fields_from_grid	Compute derivative fields on a grid and flatten them for regression.
discover_from_grid	Full discovery on gridded data: derivatives, library, sparse regression.
discover_from_scatter	Full discovery on scattered data: reconstruct, optionally smooth, then discover_from_grid.
equation_to_string	Format a discovery result as a readable equation string.
evaluate_against_ground_truth	Compare a discovered equation with the hidden truth (grading only, after discovery).
evaluate_competing_hypotheses	Fit each named single-term hypothesis separately and report R^2 and coefficients.

equation_discovery/library.py

term_label	Human-readable label for a library term, e.g. ('u', 'u_x') -> 'u*u_x'.
build_library	Evaluate every candidate term at every sample: the matrix Theta.
parse_term_specs	Convert the config's term lists into term tuples.

equation_discovery/sindy.py

stlsq	Sequentially thresholded least squares: fit, zero small coefficients, refit, repeat (columns normalised).
run_sindy	Fit $u_t = \text{Theta } \xi$ with STLSQ (pysindy if installed, else stlsq) and return a structured result.
fit_restricted	Ordinary least squares on a fixed set of terms (used for competing hypotheses).

`pinn/`

`pinn/data_utils.py`

`_to_tensor`

Convert a numpy array to a float32 column tensor.

`load_observational_data`

Load noisy.npz or sparse.npz, returning only x, t, u (the only data a PINN may train on).

`load_clean_reference`

Load the clean field for evaluation only, never for training.

`make_collocation_points`

Random (x, t) points where the physics residual is penalised.

`make_bc_points`

Random points on both boundaries with target $u = 0$.

`make_ic_points`

Random points at $t = 0$ with target $u = \text{initial condition}$.

`data_dict_to_tensors`

Convert an (x, t, u) dict into tensors.

`filter_by_time`

Keep only observations with t below a cutoff (train/OOD split).

`pinn/derivatives.py`

`compute_derivatives`

Exact u , u_t , u_x , u_{xx} (and u_{xxx}) of the network output via torch.autograd.

`pinn/forward.py`

`run_forward_pinn`

Train a PINN with physics parameters fixed (known D).

`pinn/inverse.py`

`run_inverse_pinn`

Train a PINN with physics parameters trainable (estimate D).

`pinn/losses.py`

`mse`

Mean squared error helper.

`data_loss`

Mismatch between network and observations.

`physics_loss`

Mean squared equation residual at collocation points.

`bc_loss`

Mismatch at the boundaries.

`ic_loss`

Mismatch with the initial condition.

`total_loss`

Weighted sum of the four losses.

`pinn/network.py`

`PINN`

The one reusable network: $(x, t) \rightarrow u$ through four tanh layers of 64 units.

`PINN._init_weights`

Xavier initialisation of the weights.

`PINN.forward`

Evaluate u at given (x, t) .

`pinn/residuals.py`

`diffusion_residual`

Residual $u_t - D u_{xx}$.

`advection_residual`

Residual $u_t - c u_x$.

Residual $u_t - D u_{xx} + k u$.

`reaction_diffusion_residual`

`make_params`

Wrap initial parameter values as tensors, trainable or fixed.

`pinn/trainer.py`

`PINNTrainer`

Holds network, residual function, parameters and loss weights; runs training.

`PINNTrainer.trainable_parameters`

Network weights plus any trainable physics parameters.

`PINNTrainer.compute_losses`

`PINNTrainer.train`

Adam steps then an L-BFGS polish, recording loss history.

`PINNTrainer._record`

Store one row of loss history and current parameter values.

`build_and_train`

Shared entry point: build network and parameters, train, return trainer and history.

validation/

validation/conservation.py
conservation_violation

Global balance check: does the model's total amount change exactly as its own law says ($dm/dt = \text{integral of } f$)?

validation/forward_solver.py
rhs_function
_effective_coefficients
solve_forward

Compile any candidate equation into a numerical right-hand side $u_t = f(u, u_x, u_xx)$.

Estimate diffusion/advection/reaction strengths to choose a stable time step.

Solve a candidate law forward in time from an initial condition (Euler or RK4).

validation/model_selection.py
parameter_contributions
reduced_signature
bic
select_model

Term necessity: share of $\text{RMS}(u_t)$ contributed by each parameter's terms on the trained PINN.

Signature of the equation with inactive terms removed (does it reduce to another candidate?).

Bayesian information criterion $n \ln(\text{MSE}) + k \ln n$.

The selection rule: gate, fit-equivalence, optional reduction, complexity, BIC.

validation/ood.py
split_by_time
prediction_error

Split points into $t \leq \text{cutoff}$ and $t > \text{cutoff}$.

RMSE and MAE of a model against reference values.

validation/physics_validation.py
evaluate_physics_residual

Mean squared residual of a trained PINN on a fresh grid (not the training points).

validation/scorer.py
build_validation_result

Turn measured losses into supported/rejected using fixed thresholds (Phase 3).

validation/stability.py
candidate_monomials
bootstrap_stability

The distinct field terms on a candidate's right-hand side (e.g. u_xx , u , u^2).

Refit those terms by least squares on 20 random 80% subsets; report mean, spread, sign stability.

llm/

llm/__init__.py
get_provider

Choose the LLM provider from config and environment (Gemini if a key is set, else the mock).

llm/base.py
LLMProvider
LLMProvider.generate
LLMProvider.name

Abstract interface every LLM provider implements.

Return the raw text completion for a prompt.

The provider's class name, recorded in results.

llm/gemini_provider.py
_repair_json_escapes

If the response is not valid JSON, double stray backslashes (LaTeX like `\partial`).

GeminiProvider

Real provider for Google Gemini; the key comes only from GEMINI_API_KEY.

<code>GeminiProvider._get_client</code>	Lazily import google-genai and create the client; clear error if no key.
<code>GeminiProvider.generate</code>	Call Gemini in JSON mode, retrying overload errors (429/500/503) with back-off.
<code>llm/mock_provider.py</code>	
<code>MockLLMProvider</code>	Deterministic offline provider returning a canned set of four hypotheses.
<code>MockLLMProvider.generate</code>	Return the same canned JSON every time, whatever the prompt.
<code>llm/openai_provider.py</code>	
<code>OpenAIProvider</code>	Legacy OpenAI provider (no longer selectable by <code>get_provider</code>). Lazily create the OpenAI client.
<code>OpenAIProvider._get_client</code>	
<code>OpenAIProvider.generate</code>	Call OpenAI chat completions.
<code>llm/prompts.py</code>	
<code>build_hypothesis_prompt_with_context</code>	Mode B prompt: observations plus retrieved evidence, mechanisms, equations and parameters.
<code>build_hypothesis_prompt</code>	Mode A prompt: observations only, with the JSON schema the answer must follow.
<code>llm/schemas.py</code>	
<code>ParameterSpec</code>	One parameter: name, trainable flag, initial value, units, optional bounds.
<code>Hypothesis</code>	The structured hypothesis every LLM answer must fit (pydantic model).
<code>Hypothesis._confidence_in_range</code>	Validator: confidence must lie in [0, 1].
<code>Hypothesis._equation_has_equality</code>	Validator: the equation must contain '='.
<code>HypothesisSet</code>	A validated collection of hypotheses for one problem.
hypotheses/	
<code>hypotheses/adapter.py</code>	
<code>hypothesis_to_pinn_config</code>	Compile a hypothesis into exactly what <code>build_and_train</code> needs (residual function, parameters).
<code>hypotheses/complexity.py</code>	
<code>equation_complexity</code>	Complexity score: terms + free parameters + derivative order + 2 x nonlinear + 0.1 x operations.
<code>hypotheses/constraints.py</code>	
<code>resolve_bounds</code>	Combine a hypothesis's declared bounds with the config fallbacks ($D > 0$, $K > 0$).
<code>precheck_physics</code>	Before training: reject non-evolution equations; warn on over-determined or uncoupled boundary setups.
<code>check_fitted_bounds</code>	After training: flag fitted parameters that violate physical bounds.
<code>hypotheses/equation_compiler.py</code>	
<code>EquationParseError</code>	Raised when an equation string cannot be parsed.
<code>parse_equation</code>	Parse 'lhs = rhs' with SymPy into a residual; detect fields and parameters used.
<code>compile_residual</code>	Turn the residual into a torch function <code>residual_fn(derivs, params)</code> for the trainer.
<code>equation_str_to_sympy_str</code>	Normalised human-readable form of an equation.

<code>hypotheses/pipeline.py</code>	
<code>_extract_json_array</code>	Strip markdown code fences and parse the LLM's JSON array.
<code>generate_hypotheses</code>	Prompt, call the provider, parse, schema-check, validate, deduplicate and rank.
<code>hypotheses/ranking.py</code>	
<code>_completeness_score</code>	Fraction of descriptive fields the LLM filled in.
<code>_simplicity_score</code>	$1 / (1 + \text{number of parameters})$.
<code>rank_hypotheses</code>	Composite score that sets investigation order only, never the verdict.
<code>hypotheses/search.py</code>	
<code>kg_template_candidates</code>	Turn equations documented in the knowledge graph into candidate hypotheses.
<code>build_candidate_pool</code>	Merge LLM and graph candidates and remove structural duplicates.
<code>structural_and_physics_filter</code>	Run deterministic validation and physics pre-checks; keep the survivors.
<code>make_splits</code>	Train / random holdout / OOD ($t > \text{cutoff}$) splits of the observations.
<code>_rmse</code>	RMSE of a network on a split.
<code>validate_candidate</code>	Inverse PINN on the training split, then gate checks, bounds, term necessity, complexity, BIC.
<code>load_candidate_model</code>	Reload a saved candidate network.
<code>train_data_only_mlp</code>	Baseline: the same network trained on data only, without physics.
<code>save_json</code>	Write a result dict to JSON, creating folders as needed.
<code>hypotheses/validation.py</code>	
<code>validate_hypothesis</code>	Deterministic checks: syntax, variables, parameters, derivatives, units.
<code>canonical_signature</code>	Parameter- and sign-invariant structure of an equation, for duplicate detection.
<code>deduplicate</code>	Remove hypotheses with identical canonical signatures.
<code>rag/</code>	
<code>rag/chunker.py</code>	
<code>chunk_document</code>	Split a document into chunks at its section headers.
<code>rag/context.py</code>	
<code>ScientificContext</code>	The unified evidence object: retrieved evidence, mechanisms, equations, parameters, sources.
<code>build_scientific_context</code>	Retrieve evidence for the description, then query the graph for every mechanism it mentions.
<code>rag/corpus_loader.py</code>	
<code>_parse_frontmatter</code>	Split a Markdown file into its metadata header and body.
<code>load_document</code>	Load one corpus file as a Document.
<code>load_corpus</code>	Load every .md file in rag/corpus/.
<code>rag/documents.py</code>	
<code>DocumentMetadata</code>	A document's id, title, source, domain and declared mechanism, equations and parameters.
<code>Document</code>	Metadata plus text.
<code>Chunk</code>	A piece of a document with its section and id.
<code>rag/embeddings.py</code>	
<code>EmbeddingProvider</code>	Interface for turning text into vectors.
<code>EmbeddingProvider.embed</code>	Embed one text.

<code>EmbeddingProvider.embed_many</code>	Embed many texts.
<code>EmbeddingProvider.name</code>	Provider name.
<code>LocalEmbeddingProvider</code>	Deterministic offline embedding: words hashed into 256 slots, normalised.
<code>LocalEmbeddingProvider._token_index</code>	Hash a word to a slot (MD5 modulo 256).
<code>LocalEmbeddingProvider.embed</code>	Count words per slot and normalise to unit length.
<code>APIEmbeddingProvider</code>	Optional real embedding API provider (unused by default).
<code>APIEmbeddingProvider._get_client</code>	Lazily create its API client.
<code>APIEmbeddingProvider.embed</code>	Embed via the API.
<code>rag/evidence.py</code>	
<code>RetrievalResult</code>	One retrieved chunk with its similarity score.
<code>Evidence</code>	A retrieved claim with provenance; its confidence is retrieval confidence only.
<code>evidence_from_retrieval</code>	Convert a retrieval result into an Evidence object.
<code>rag/metadata.py</code>	
<code>matches_filters</code>	Check a chunk's metadata against filter values.
<code>rag/pipeline.py</code>	
<code>RAGPipeline</code>	Ties chunker, embedder and vector store together.
<code>RAGPipeline.ingest</code>	Chunk, embed and store every document.
<code>RAGPipeline.get_retriever</code>	Return a Retriever over the stored chunks.
<code>rag/retriever.py</code>	
<code>Retriever</code>	Search interface over the vector store.
<code>Retriever.retrieve</code>	Embed the query, take the most similar chunks, apply filters, return Evidence.
<code>rag/vector_store.py</code>	
<code>VectorStore</code>	Interface: add, search, get.
<code>VectorStore.add</code>	Store a vector with its payload.
<code>VectorStore.search</code>	Return the top-k most similar vectors.
<code>VectorStore.get</code>	Fetch one stored item.
<code>InMemoryVectorStore</code>	Brute-force cosine-similarity store in memory.
<code>InMemoryVectorStore.add</code>	Store a vector with its payload.
<code>InMemoryVectorStore.search</code>	Cosine similarity against every stored vector; return the top-k.
<code>InMemoryVectorStore.get</code>	Fetch one stored item.
<code>InMemoryVectorStore.all_ids</code>	List stored ids.
<code>PersistentVectorStore</code>	The in-memory store plus JSON save/load.
<code>PersistentVectorStore.save</code>	Write the store to JSON.
<code>PersistentVectorStore.load</code>	Read the store from JSON.
<code>knowledge_graph/</code>	
<code>knowledge_graph/builder.py</code>	

<code>_slug</code>	Make a stable id fragment from a name.
<code>_find_chunk_id</code>	Find the chunk that states a fact (provenance).
<code>build_graph_from_documents</code>	Create Document, Mechanism, Equation, Variable and Parameter entities and sourced relationships.
knowledge_graph/graph.py	
<code>_relationship_id</code>	Stable hash of (subject, predicate, object), so repeated facts merge.
<code>KnowledgeGraph</code>	The graph container (backed by networkx).
<code>KnowledgeGraph.add_entity</code>	Add an entity, merging if its id exists.
<code>KnowledgeGraph.get_entity</code>	Look up an entity by id.
<code>KnowledgeGraph.entities_</code>	All entities of one type.
by_type	
<code>KnowledgeGraph.add_</code>	Add a typed, sourced edge, merging sources for duplicates.
relationship	
<code>KnowledgeGraph.neighbors</code>	Entities connected to a given entity.
	Edges touching a given entity.
<code>KnowledgeGraph.relationships_</code>	
for	
<code>KnowledgeGraph.summary</code>	Counts of entities and relationships by type.
knowledge_graph/query.py	
<code>find_entity_by_name</code>	Find an entity by its name.
<code>equations_for_mechanism</code>	Documented equations that govern a mechanism.
<code>variables_for_mechanism</code>	Variables in those equations.
<code>parameters_for_mechanism</code>	Parameters in those equations.
	Mechanisms linked to an observation entity (schema-ready).
mechanisms_for_observation	
<code>documents_supporting</code>	Documents that support an entity.
<code>sources_for_relationship</code>	Provenance of an edge.
<code>evidence_connecting</code>	Relationships and shared equations linking two variables.
knowledge_graph/schema.py	
<code>EntityType</code>	Allowed entity types (Document, Mechanism, Variable, Parameter, Equation, Observation).
<code>RelationType</code>	Allowed relationship types: GOVERNS, HAS_VARIABLE, HAS_PARAMETER, SUPPORTED_BY, CAUSES, DEPENDS_ON, and others.
<code>Entity</code>	A node: id, type, name, metadata.
<code>SourceRef</code>	Where a fact came from: document and chunk.
<code>Relationship</code>	A typed edge with its sources.
knowledge_graph/serialization.py	
<code>graph_to_dict</code>	Convert the graph to a JSON-able dict.
<code>graph_from_dict</code>	Rebuild a graph from that dict.
<code>save_graph</code>	Write the graph to a JSON file.
<code>load_graph</code>	Read a graph from a JSON file.
evaluation/	
evaluation/ablation.py	
<code>_discovery_only</code>	The SINDy-alone arm: discover, then match the result to a documented mechanism.
<code>run_ablation</code>	Run all seven arms on the 12 datasets.
<code>_median</code>	Median helper.

<code>summarize_ablation</code>	Per-arm totals: correct model, wrong candidates rejected, parameter and OOD accuracy.
<code>evaluation/adversarial.py</code>	
<code>_CannedProvider</code>	Test LLM that returns a fixed (possibly malicious) response.
<code>_CannedProvider.generate</code>	Return the canned response.
<code>_H</code>	Shorthand to build a test Hypothesis.
<code>_case</code>	Record one adversarial case result.
<code>find_leaks</code>	Search corpus text for any hidden benchmark value.
<code>run_adversarial</code>	Run the 20 attack cases.
<code>evaluation/benchmark.py</code>	
<code>_item</code>	Record one question result.
<code>_hyp</code>	Build a Hypothesis from an equation string, inferring parameter names.
<code>run_single_questions</code>	The 50 single questions over the 12 datasets and the graph.
<code>run_multihop</code>	The 25 multi-hop chains.
<code>evaluation/datasets.py</code>	
<code>Dataset</code>	One benchmark system: family, truth, grids, field, initial condition, exact solution.
<code>_diffusion</code>	Exact two-mode diffusion dataset.
<code>_gauss</code>	Exact moving, spreading Gaussian; used for advection ($D = 0$) and advection-diffusion.
<code>_advection</code>	Pure advection dataset.
<code>_adv_diff</code>	Advection-diffusion dataset.
<code>_reaction_diffusion</code>	Reaction-diffusion dataset solved numerically with RK4.
<code>benchmark_datasets</code>	The 12 datasets (4 families x 3 settings), cached to .npz.
<code>_build_all</code>	Construct all 12 datasets.
<code>_rd_shell</code>	Reaction-diffusion dataset without its (expensive) field, for quick metadata.
<code>add_noise</code>	Add Gaussian noise to a dataset's field.
<code>evaluation/describe.py</code>	
<code>_centroid</code>	Amplitude-weighted centre of a profile.
<code>_spread</code>	Width of a profile.
<code>_n_peaks</code>	Number of peaks in a profile.
<code>observation_stats</code>	Amplitude change, drift, spread and peaks between early and late times.
<code>describe_observations</code>	Fixed-template text description from those statistics (never names a mechanism).
<code>evaluation/engine.py</code>	
<code>Resources</code>	Corpus, RAG index and graph, built once and shared.
<code>build_pool</code>	Description -> RAG/graph context -> candidate pool (evidence equations, graph templates, mock LLM).
<code>_grid_param_contributions</code>	Term necessity computed on grid derivatives instead of a PINN.
<code>_reduced_term_set</code>	Term set of an equation with its inactive terms removed.
<code>validate_and_select</code>	Fast pipeline: least-squares fit per candidate, gate, selection.
<code>predict_ood</code>	Solve the selected law forward beyond the data window.
<code>family_of</code>	Map a hypothesis's term set to its mechanism family.
<code>truth_field</code>	Ground-truth field at given times (evaluation only).
<code>score_run</code>	Score one run: correct family, parameter error, OOD error.
<code>family_of_row</code>	Mechanism family from a stored result row.
<code>evaluation/identify.py</code>	
<code>_syms</code>	SymPy symbols for the fields.

<code>term_set</code>	The set of field terms in a hypothesis.
<code>_label_to_expr</code>	Convert a library label like 'u*u_x' to a SymPy expression.
<code>discovered_coefficients</code>	Non-zero SINDy coefficients as {term: value}.
<code>match_mechanism</code>	Find the documented template with exactly the discovered term set.
<code>recover_parameters</code>	Solve for a template's parameters from discovered coefficients.
<code>grid_fields</code>	Finite-difference fields of a gridded dataset, flattened.
<code>fit_candidate_on_grid</code>	Least-squares fit of u_t on a candidate's own terms; train and holdout error.
<code>library_collinearity</code>	Identifiability check: maximum correlation between library columns.
evaluation/leakage.py	
<code>ground_truth_references</code>	List ground-truth identifiers mentioned in a function's source code.
<code>decision_functions</code>	The six functions that make decisions (pool, validation, splits, selection).
<code>leakage_audit</code>	Static audit of those functions plus a corpus scan for hidden values.

experiments/

experiments/02_equation_discovery.py

<code>load_config</code>	Read config.yaml.
<code>run_case</code>	Run discovery on one dataset variant and evaluate it.
<code>main</code>	Phase 2: clean, sparse, noisy (raw and smoothed); save report, table and plot.

experiments/03_forward_pinn.py

<code>load_config</code>	Read config.yaml.
<code>main</code>	Phase 3 forward PINN with $D = 0.1$ fixed.

experiments/04_inverse_pinn.py

<code>load_config</code>	Read config.yaml.
<code>main</code>	Phase 3 inverse PINN estimating D from 0.2.

experiments/05_hypothesis_validation.py

<code>load_config</code>	Read config.yaml.
<code>run_hypothesis</code>	Train one hypothesis's PINN and score it.
<code>main</code>	Phase 3: diffusion vs advection with a time-based OOD split.

experiments/06_hypothesis_generation.py

<code>load_config</code>	Read config.yaml.
<code>main</code>	Phase 4: generate, validate and rank hypotheses; optionally run PINNs.

experiments/07_scientific_rag_kg.py

<code>load_config</code>	Read config.yaml.
<code>hyp_to_dict</code>	Serialise a hypothesis.
<code>summarize_generation</code>	Counts for a generation run (candidates, valid, duplicates, evidence-supported).
<code>main</code>	Phase 5: corpus, RAG, graph, context, leakage check, Mode A vs Mode B.

experiments/08_physics_constrained_search.py

<code>load_config</code>	Read config.yaml.
<code>stage_prepare</code>	Phase 6 stage 1: context, generation, pool, filters.
<code>_load_survivors</code>	Reload the filtered candidates.
<code>_splits</code>	Build train/holdout/OOD splits from noisy.npz.
<code>_setup</code>	Domain and initial-condition settings for the PINN.

<code>stage_pinn</code>	Phase 6 stage 2: inverse PINN per survivor, cached.
<code>oracle_evaluation</code>	Benchmark-only error against the hidden truth, after selection.
<code>stage_select</code>	Phase 6 stage 3: stability, selection, baselines, OOD, novel prediction, report.
<code>_plots</code>	Draw the Phase 6 summary figure.
<code>_report</code>	Write <code>phase6_report.md</code> .
<code>main</code>	Run the chosen stages.
experiments/09_evaluation.py	
<code>save</code>	Write one Phase 7 result file.
<code>real_llm</code>	Mode A vs Mode B with a real LLM on 3 datasets (only with <code>--real-llm</code> and a key).
<code>acc</code>	Format an accuracy fraction.
<code>report</code>	Write <code>phase7_report.md</code> .
<code>main</code>	Phase 7: questions, multi-hop, adversarial, ablation, leakage, determinism check, report.
experiments/10_generalization.py	
<code>_truth_dataset</code>	The hidden advection-diffusion system ($c = 0.4$, $D = 0.02$).
<code>observations</code>	400 noisy samples, the only data the pipeline sees.
<code>_recon</code>	Grid reconstruction for SINDy and the description.
<code>_setup</code>	Domain settings for the PINN.
<code>stage_prepare</code>	SINDy baseline, pool and filters for the new system.
<code>_survivors</code>	Reload filtered candidates.
<code>_splits</code>	Train/holdout/OOD splits.
<code>stage_pinn</code>	Inverse PINN per candidate.
<code>stage_select</code>	Selection under both rules, OOD, oracle evaluation.
<code>main</code>	Run the chosen stages.
experiments/11_real_llm_gemini.py	
<code>_load_script</code>	Import an experiment script by file path.
<code>_phase6_module</code>	Load the Phase 6 script and redirect its outputs to the Gemini folder.
<code>stage_generate</code>	Run 09's <code>real_llm</code> Mode A vs Mode B with Gemini.
<code>annotate_report</code>	Mark the reused Phase 6 report as a real-LLM run.
<code>compare_with_mock</code>	Side-by-side summary of the mock and Gemini Phase 6 runs.
<code>main</code>	Check the key, force the Gemini provider, run the chosen stages.
experiments/12_conservation.py	
<code>_windows</code>	Conservation violation in the train, OOD and beyond-data windows.
<code>evaluate_run</code>	Conservation check for every saved candidate of one Phase 6 run (no retraining).
<code>main</code>	Check the mock and Gemini runs plus the physics-free baseline; write <code>results/evaluation/conservation/</code> .

B Running the project yourself

All commands run from the project root (`scientific-discovery-engine/`).

Install and test

```
python -m pip install -r requirements.txt
python -m pytest tests -q          # 94 tests, no API key or network needed
```

Reproduce each phase

Command	Time	Writes to
python data/generate_diffusion.py	seconds	data/generated/
python experiments/02_equation_discovery.py	seconds	results/equations/
python experiments/03_forward_pinn.py (and 04, 05)	~4 min each	results/pinn/
python experiments/06_hypothesis_generation.py --skip-pinn	seconds	results/hypotheses/
python experiments/07_scientific_rag_kg.py	seconds	results/rag_kg/
python experiments/08_physics_constrained_search.py	~20 min	results/phase6/
python experiments/10_generalization.py	~16 min	results/evaluation/generalization/
python experiments/09_evaluation.py	~1 min	results/evaluation/

Every script accepts `--config config.yaml`. The PINN experiments accept `--stage` and `--only` and cache each candidate, so they can be resumed.

Watch out: the configured provider is Gemini

`config.yaml` now says `provider: "gemini"`. If `GEMINI_API_KEY` is exported in your shell, scripts 06, 07 and 08 will call Gemini and *overwrite* the mock results in their folders. To reproduce the recorded mock results, leave the key unset (they fall back to the mock with a warning) or set `provider: "mock"`.

The real Gemini run

```
set -a; source .env; set +a          # loads GEMINI_API_KEY without printing it
python experiments/11_real_llm_gemini.py --stage generate \
    --model gemini-3.5-flash-lite
python experiments/11_real_llm_gemini.py --stage prepare \
    --model gemini-3.5-flash-lite
python experiments/11_real_llm_gemini.py --stage pinn      # ~25 min
python experiments/11_real_llm_gemini.py --stage select
```

Outputs go only to `results/evaluation/real_llm_gemini/`.

Rebuilding this guide

```
python docs/gen_function_reference.py # refresh Appendix A from the code
cd docs && tectonic project_guide.tex # build project_guide.pdf
```

The generator refuses to run if any function in the code lacks a description, so the reference cannot silently go out of date.

Where to read more

`README.md` (summary and the development log written during each phase), `results/final_report.md` (the audited final report), `results/demo/demo_run.md` (a guided demo), and the mock-vs-Gemini comparison in `results/evaluation/real_llm_gemini/ comparison_with_mock.json`.